

算法设计与分析

Design & Analysis of Algorithms

第5章 回溯法

第5章 回溯法

5.1 回溯法概述

5.2 求解0/1背包问题

5.3 求解装载问题

5.4 求解子集和问题

5.5 求解 n 皇后问题

5.6 求解图的 m 着色问题

5.7 求解旅行商问题

5.8 求解活动安排问题

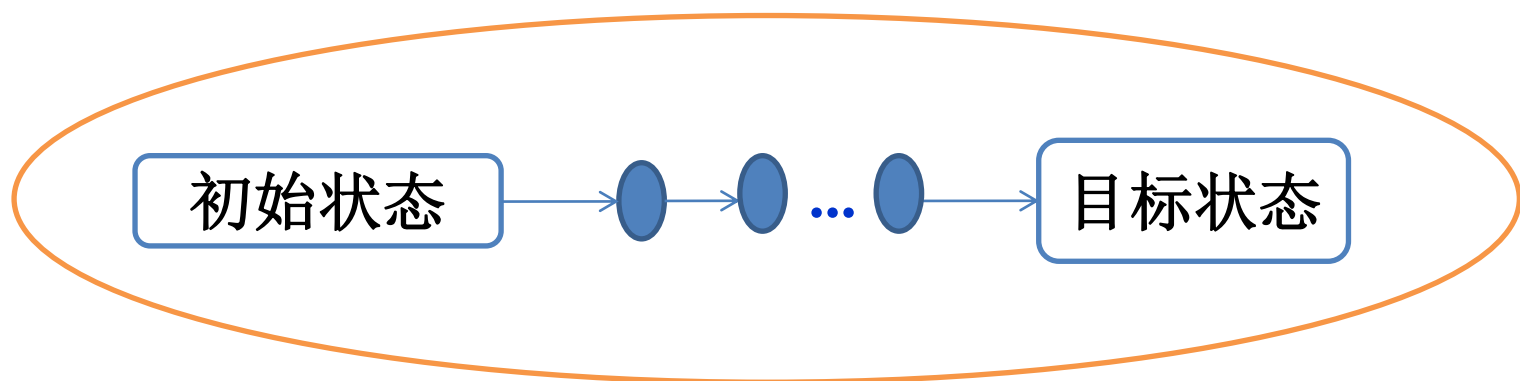
5.9 求解流水作业调度问题

5.1 回溯法概述

- 回溯法，又称为**试探法**。类似穷举的搜索尝试过程，主要是在搜索尝试过程中寻找问题的解，当发现已不满足求解条件时，就“回溯”（即回退），尝试别的路径。
- 满足回溯条件的某个状态的点称为“回溯点”。

5.1.1 问题的解空间

- 一个复杂问题的解决方案是由若干个小的决策步骤组成的决策序列，解决一个问题的所有可能的决策序列构成该问题的解空间。



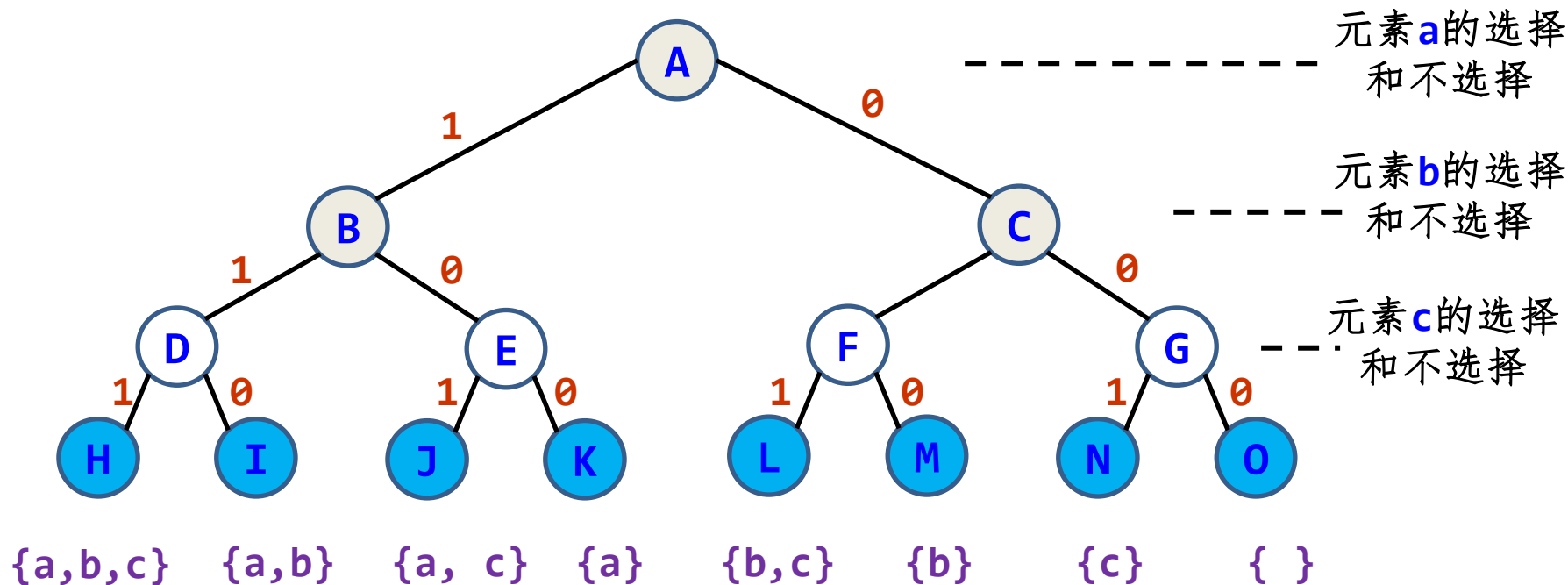
- 应用回溯法求解问题时，首先应该明确问题的解空间。解空间中满足约束条件的决策序列称为可行解。
- 一般来说，解任何问题都有一个目标，在约束条件下使目标达到最优的可行解称为该问题的最优解。

- 问题的解由一个不等长或等长的解向量 $X=\{x_1, x_2, \dots, x_n\}$ 组成，其中分量 x_i 表示第 i 步的操作，即第 i 步的选择
- 所有满足约束条件的解向量组构成了问题的解空间。
- 问题的解空间一般用树形式来组织，也称为解空间树或状态空间，树中的每一个结点确定所求解问题的一个问题状态。
- 树的根结点位于第1层，表示搜索的初始状态，第2层的结点表示对解向量的第一个分量做出选择后到达的状态，以此类推。

例5.1, 求集合 $\{a, b, c\}$ 的幂集的解空间树:

- 解向量 $X=(x_1, x_2, x_3)$, $x_i=1$ 表示选择对应元素, $x_i=0$ 表示不选择该元素
- 求解过程分为3步:
 1. 分别对 a 、 b 、 c 三个元素做决策, 选择或者不选择
 2. 每个叶结点都构成一个解 (很多情况并非如此)
 3. 每个非叶结点对应一个部分解向量

例5.1，求集合 $\{a, b, c\}$ 的幂集的解空间树：



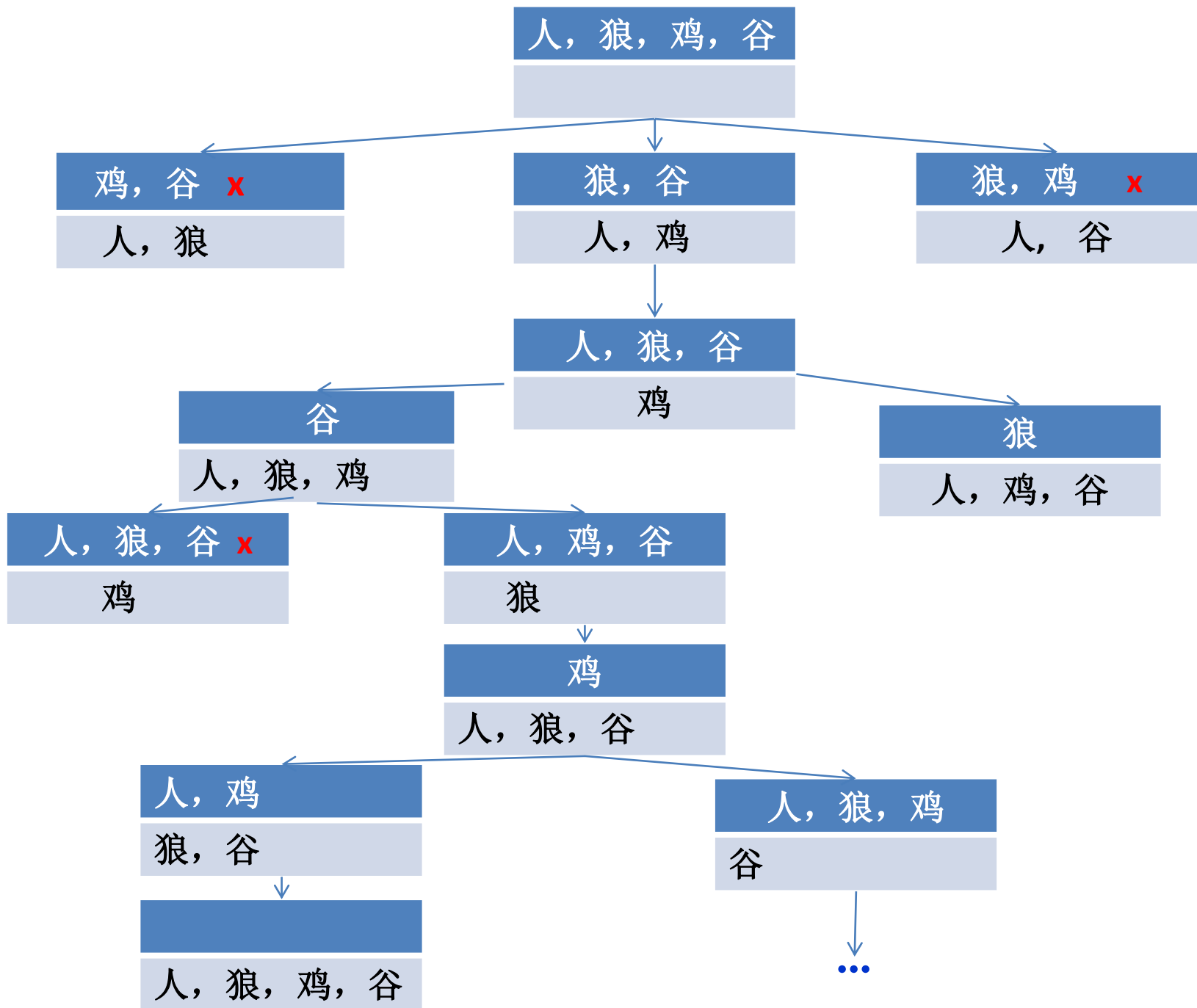
- 在一个解空间中搜索解的过程构成搜索空间，图中所有叶子结点都是解，所以该问题的解空间和搜索空间相同。

■ 一个问题求解过程就是在对应的解空间中搜索以满足目标函数的解，所以算法设计的关键点有3个：

- 结点是如何扩展的，例如求幂集的问题中，第 i 层结点的扩展方式就是选择 a_i 和不选择两种，但有些问题扩展结点是很复杂的。
- 在解空间树中，按什么方式搜索，一种是深度优先遍历（DFS），回溯法就是这种，还有一种是广度优先遍历（BFS），如分枝限界法。
- 解空间树通常是十分庞大的，如何高效的找到问题的解。

例5.2 农夫过河问题：东岸一个农夫、一只狼、一只鸡和一袋谷子。只有当农夫在现场时，狼不会把鸡吃掉，鸡也不会吃谷子，否则会现这样的情况。

另有一条小船，该船只能由农夫操作，且最多只能载下农夫和另一样东西。设计一种过河方案，将农夫、狼、鸡和谷子借助小船运到河西岸。



子集树与排列树

■ 子集树

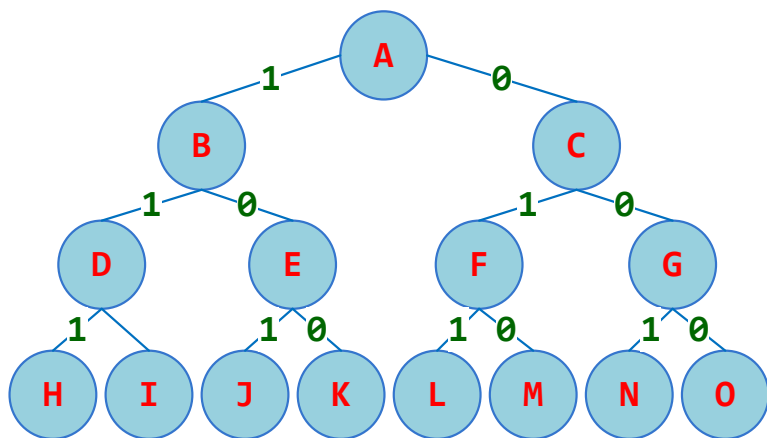
- 从 n 个元素的集合 S 中找出**满足某种性质的子集**，相应的解空间称为**子集树**。通常有 2^n 个叶结点，结点总数为 $2^{n+1}-1$ 。需 $\Omega(2^n)$ 计算时间。
- 如 n 个物品的**0-1背包问题**的解空间是一棵**子集树**。

■ 排列树

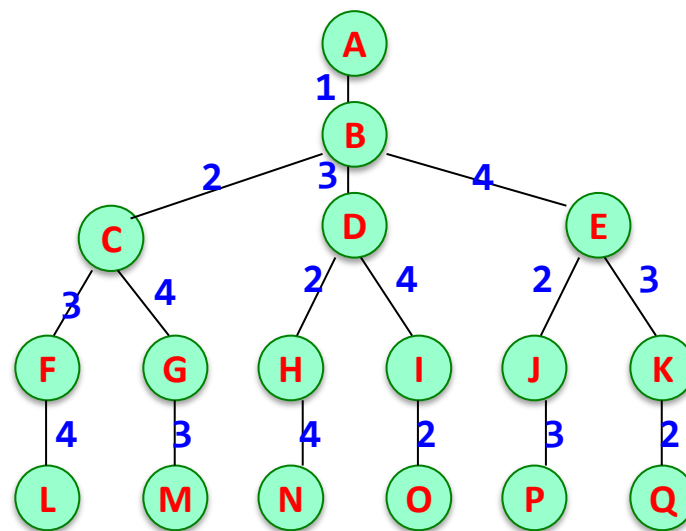
- 当所给问题的确定 n 个元素**满足某种性质的排列时**，相应的解空间树称为**排列树**。排列树通常有 $n!$ 个叶结点。因此遍历排列树需要 $\Omega(n!)$ 计算时间。
- **TSP(旅行售货员问题)**的解空间是一棵排列树。

子集树与排列树

子集树



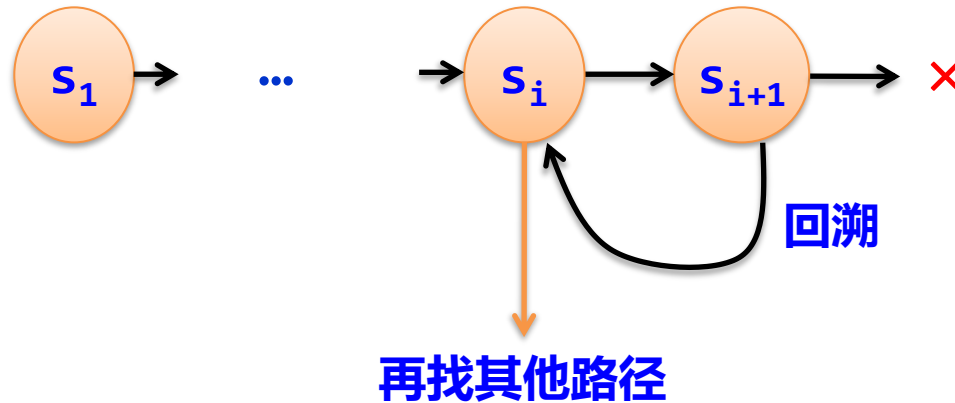
排列树



5.1.2 什么是回溯法

- 在包含问题的所有解的解空间树中，按照**深度优先搜索**的策略，从根结点（开始结点）出发搜索解空间树。
- 首先根结点成为**活结点**（活结点是指自身已生成但其孩子结点没有全部生成的结点），同时也成为当前的**扩展结点**（扩展结点是指正在产生孩子结点的结点）。
- 在当前的扩展结点处，搜索向纵深方向移至一个新结点。这个新结点就成为新的活结点，并成为**当前扩展结点**。
- 如果在当前的扩展结点处不能再向纵深方向移动，则当前扩展结点就成为**死结点**（死结点是指由根结点到该结点构成的部分解不满足约束条件，或者其子结点已经搜索完毕）。
- 此时应往回移动（**回溯**）至最近的一个活结点处，并使这个活结点成为当前的扩展结点。
- 回溯法以这种方式递归地在解空间中搜索，直至找到所要求的解或解空间中已无活结点为止。

当从状态 s_i 搜索到状态 s_{i+1} 后，如果 s_{i+1} 变为死结点，则从状态 s_{i+1} 回退到 s_i ，再从 s_i 找其他可能的路径，所以回溯法体现出走不通就退回再走的思路。



- 若用回溯法求问题的所有解时，需要回溯到根结点，且根结点的所有可行的子树都要已被搜索完才结束。
- 若使用回溯法求任何一个解时，只要搜索到问题的一个解结束。
- 回溯法搜索解空间时，通常采用两种策略避免无效搜索，提高回溯的搜索效率：
 - 用约束函数在扩展结点处剪除不满足约束的子树；
 - 用限界函数剪去得不到问题解或最优解的子树。
- 这两类函数统称为剪枝函数。

- 用回溯法解题的一般步骤如下：

- ① 针对所给问题，确定问题的解空间树，问题的解空间树应至少包含问题的一个（最优）解。
- ② 确定结点的扩展搜索规则。
- ③ 以深度优先方式搜索解空间树，并在搜索过程中可以采用剪枝函数来避免无效搜索。

5.1.3 回溯法的算法框架

- 设问题的解是一个 n 维向量 $(x_1, x_2, \dots, x_n)$ ，约束条件是 x_i 满足某种条件，记为 $\text{constrain}(x_i)$ ；限界函数 x_i 应满足某种条件，记为 $\text{bound}(x_i)$
- 回溯法的算法通常分为非递归回溯框架和递归回溯框架两种

1. 非递归回溯框架

```
int x[n];  
void backtrack(int n)  
{  int i=1;  
  while (i>=1)  
  {  if(ExistSubNode(t))  
    {  for (j=下界;j<=上界;j++)  
      {  x[i]取一个可能的值;  
        if (constraint(i) && bound(i))  
          {  if (x是一个可行解)  
              输出x;  
            else  i++;  
          }  
        }  
      }  
    else  i--;  
  }  
}
```

//x存放解向量，全局变量
//非递归框架
//根结点层次为1
//尚未回溯到头
//当前结点存在子结点
//对于子集树，j=0到1循环
//x[i]满足约束条件或界限函数
//进入下一层次
//回溯：不存在子结点，返回上一层

2. 递归的算法框架

- 回溯法是对解空间的深度优先搜索，因为递归算法中的形参具有自动回退（回溯）的能力，所以许多用回溯法设计的算法，都设计成递归算法，比同样的非递归算法设计起来更加简便。
- 其中， i 为搜索的层次（深度），通常从0或者1开始
- 下面讨论解空间为子集树和排列树的两种情况。

(1) 解空间为子集树

```
int x[n];  
void backtrack(int i)    //x存放解向量, 全局变量  
{  if(i>n)              //求解子集树的递归框架  
    输出结果;           //搜索到叶子结点, 输出一个可行解  
    else  
    {  for (j=下界; j<=上界; j++)  //用j枚举i所有可能的路径  
        {  x[i]=j;                //产生一个可能的解分量  
            ...                    //其他操作  
            if (constraint(i) && bound(i))  
                backtrack(i+1);    //满足约束条件和限界函数, 继续下一层  
        }  
    }  
}
```

- **注意:**

1. i 从1开始调用上述算法框架, i 也可以从0开始, 只需要将 $\text{if}(i > n)$ 改为 $\text{if}(i \geq n)$
2. 在上述框架中, 通过for循环枚举了 i 的所有可能路径, 如果扩展路径只有2条, 则改为两次递归调用, 如求解0/1背包问题, 子集和问题等
3. 这里的回溯框架只有 i 一个参数, 实际应用中, 可根据具体情况设置多个参数

例5.3: 有一个含 n 个整数的数组 a ，所有元素均不相同，设计一个算法求其所有子集（幂集）。

例如， $a[]=\{1, 2, 3\}$ ，所有子集是： $\{\}$ ， $\{3\}$ ， $\{2\}$ ， $\{2, 3\}$ ， $\{1\}$ ， $\{1, 3\}$ ， $\{1, 2\}$ ， $\{1, 2, 3\}$ （输出顺序无关）。

解: 显然本问题的解空间为子集树，每个元素只有两种扩展，要么选择，要么不选择。

采用深度优先搜索思路。解向量为 $x[]$ ， $x[i]=0$ 表示不选择 $a[i]$ ， $x[i]=1$ 表示选择 $a[i]$ 。

用 i 扫描数组 a ，也就是说问题的初始状态是（ $i=0$ ， x 的元素均为0），目标状态是（ $i=n$ ， x 为一个解）。从状态（ i ， x ）可以扩展出两个状态：

- 不选择 $a[i]$ 元素 $\Rightarrow$ 下一个状态为（ $i+1$ ， $x[i]=0$ ）。
- 选择 $a[i]$ 元素 $\Rightarrow$ 下一个状态为（ $i+1$ ， $x[i]=1$ ）。

```

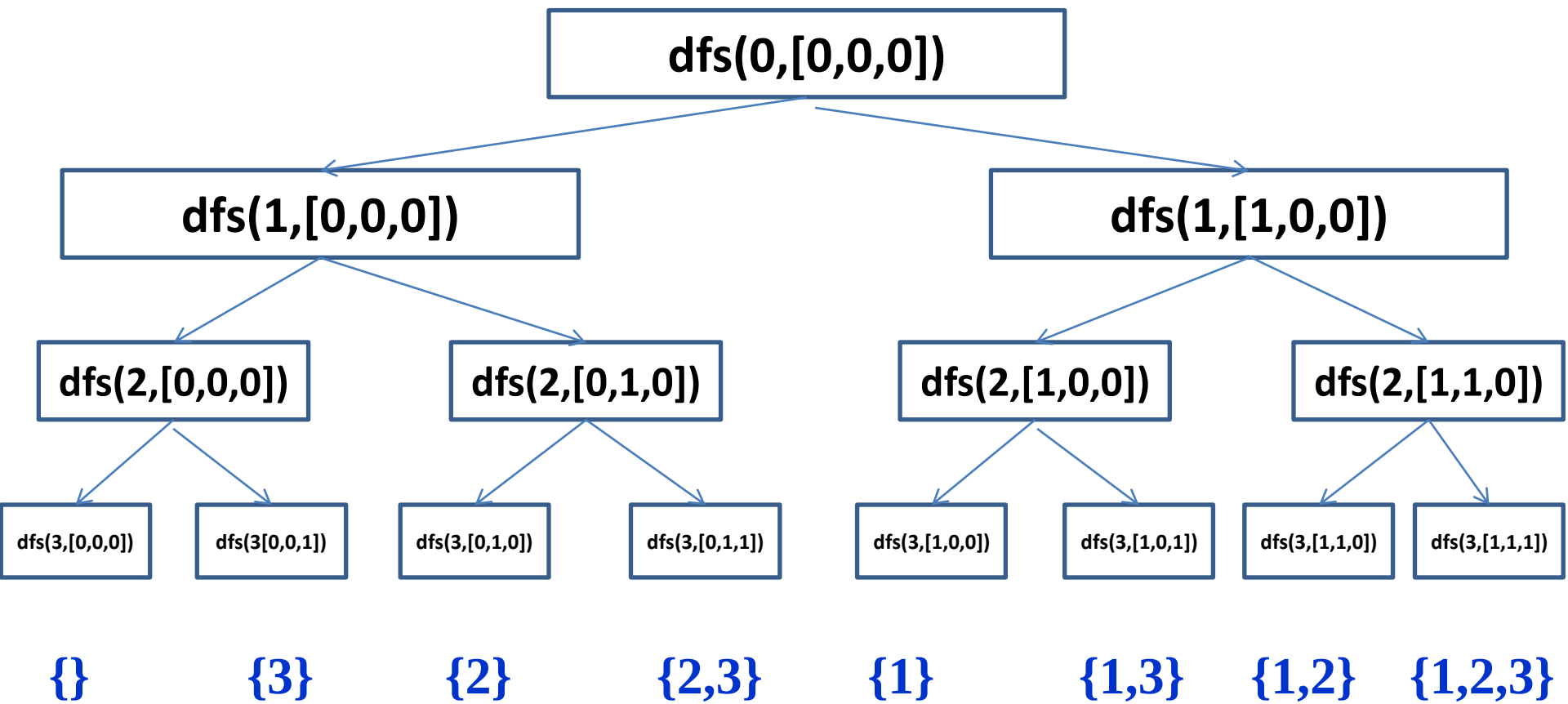
void dfs(int a[],int n,int i,int x[])
//回溯算法求解向量x
{  if (i>=n)
        dispasolution(a,n,x);           //输出一个解
    else
    {  x[i]=0; dfs(a,n,i+1,x);           //不选择a[i]
        x[i]=1; dfs(a,n,i+1,x);         //选择a[i]
    }
}

```

```

void main( )
{  int a[]={1,2,3}           //输出一个解
    int n=sizeof(a)/sizeof(a[0]);
    int x[MAXN];             //解向量
    memset(x,0,sizeofz(x));  //解向量初始化
    printf("求解结果\n");
    dfs(a,n,0,x);
    printf("\n");
}

```



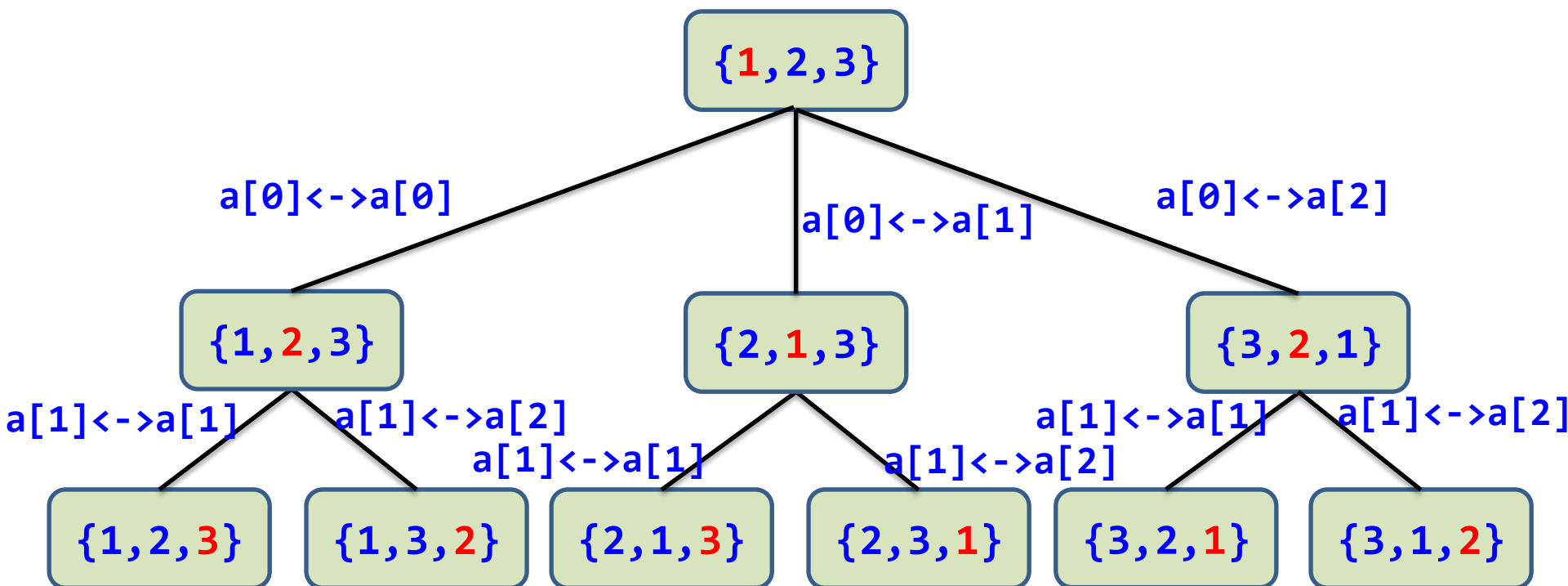
(2) 解空间为排列树

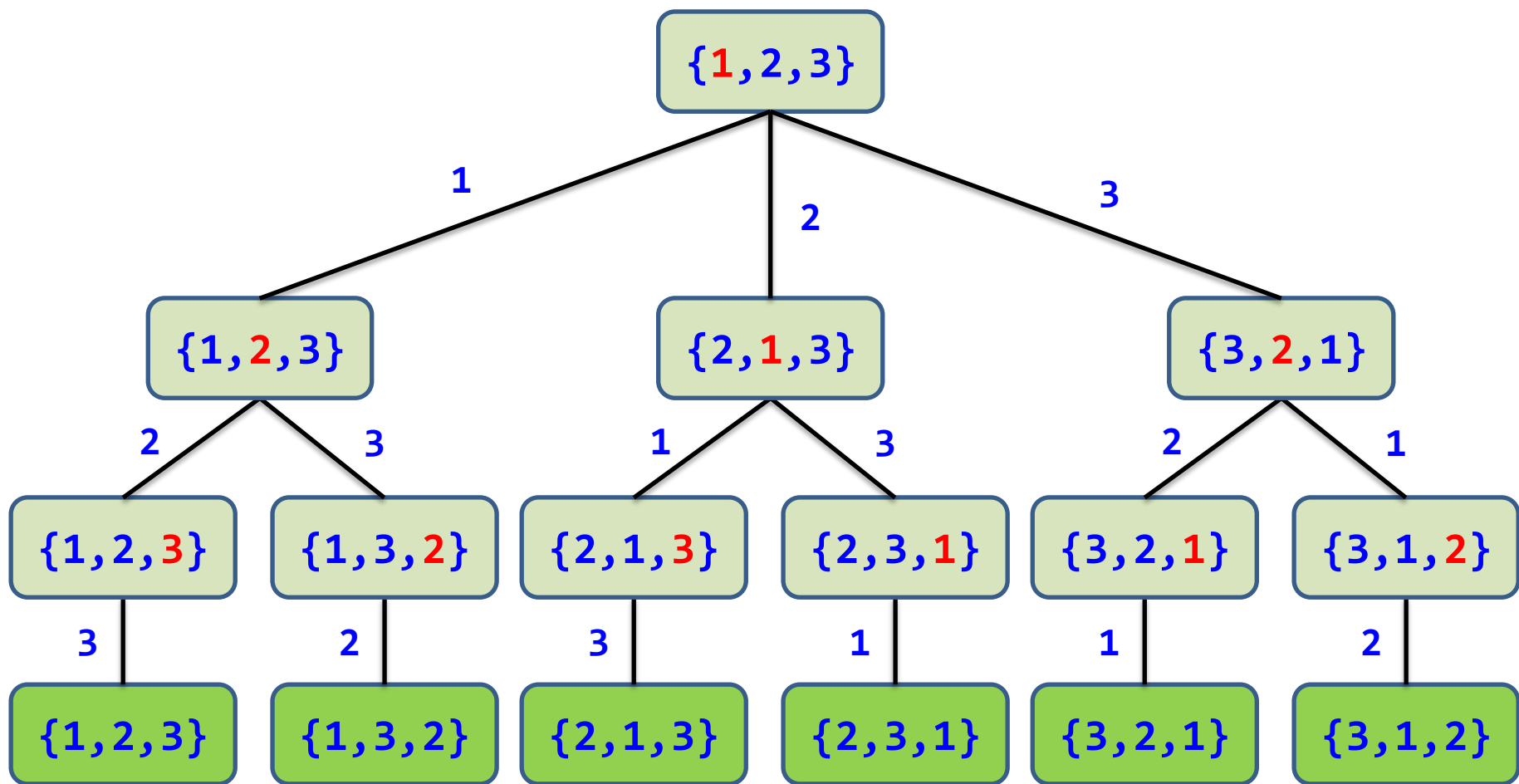
```
int x[n];  
void backtrack(int i)  
{  if(i>n)  
    输出结果;  
    else  
    {  for (j=i;j<=n;j++)  
        {  ...  
            swap(x[i],x[j]);  
            if (constraint(i) && bound(i))  
                backtrack(i+1);  
            swap(x[i],x[j]);  
            ...  
        }  
    }  
}
```

//x存放解向量, 并初始化
//求解排列树的递归框架
//搜索到叶子结点, 输出一个可行解
//用j枚举i所有可能的路径
//第i层的结点选择x[j]的操作
//为保证排列中每个元素不同, 通过交换来实现
//满足约束条件和限界函数, 进入下一层
//恢复状态
//第i层的结点选择x[j]的恢复操作

例5.4: 有一个含 n 个整数的数组 a ，所有元素均不相同，求其所有元素的全排列。例如， $a[]=\{1, 2, 3\}$ ，得到结果是 $(1, 2, 3)$ 、 $(1, 3, 2)$ 、 $(2, 3, 1)$ 、 $(2, 1, 3)$ 、 $(3, 1, 2)$ 、 $(3, 2, 1)$ 。

- 例如， $a=\{1, 2, 3\}$ ，的层次为0，它的子树分别为对应位置选择 $a[0]$ 、 $a[1]$ 、 $a[2]$ 元素。
- 采用元素交换的方式，对排列树的第 i 层，扩展状态是 $a[i]$ 位置，可以取 $a[i]$ 到 $a[n-1]$ 的任何元素，即 $j=i$ 到 $n-1$ 的循环：将 $a[i]$ 与 $a[j]$ 交换，在这种方式下，求出排列后要恢复，即再次交换，回到之前状态（回溯），然后继续求其他排列
- 对于第 i 层的结点，仅仅考虑 $a[i]$ 及后面的元素，而不必考虑前面已经选择的元素





产生了所有的解

```
void dfs(int a[],int n,int i)
{  if (i>=n)
    dispasolution(a,n);
    else
    {  for (int j=i;j<n;j++)
        {  swap(a[i],a[j]);
            dfs(a,n,i+1);
            swap(a[i],a[j]);
        }
    }
}
```

//求a[0..n-1]的全排列

//递归出口

//交换a[i]与a[j]

//交换a[i]与a[j]: 恢复

循环横向遍历

递归纵向遍历

$m=3$
 $list=\{0,1,2,3\}$

$i=k=0$
 $swap(list[0], list[0])$
 $Perm(list, k+1, m)$

0,1,2,3

$i=k+1=1$
 $swap(list[0], list[1])$
 $Perm(list, k+1, m)$

$i=k+2=2$
 $swap(list[0], list[2])$
 $Perm(list, k+1, m)$

$i=k+3=3$
 $swap(list[0], list[3])$
 $Perm(list, k+1, m)$

$i=k=1$
 $swap(list[1], list[1])$
 $Perm(list, k+1, m)$

0,1,2,3

$i=k+1=2$
 $swap(list[1], list[2])$
 $Perm(list, k+1, m)$

1,0,2,3

$i=k+2=3$
 $swap(list[1], list[3])$
 $Perm(list, k+1, m)$

3,1,2,0

$i=k=2$
 $swap(list[2], list[2])$
 $Perm(list, k+1, m)$

0,1,2,3

$i=k+1=3$
 $swap(list[2], list[3])$
 $Perm(list, k+1, m)$

$i=k=2$
 $swap(list[2], list[2])$
 $Perm(list, k+1, m)$

0,2,1,3

$i=k+1=3$
 $swap(list[2], list[3])$
 $Perm(list, k+1, m)$

0,3,2,1

$k=3$,
 $i=$

0,1,2,3

$k=3$,
 $i=$

0,1,3,2

$k=3$,
 $i=$

0,2,1,3

$k=3$,
 $i=$

0,2,3,1

{0 1 2 3}

{0 1 3 2}

{0 2 1 3}

{0 2 3 1}

结果输出

5.1.4 回溯法与深度优先遍历的异同

- 两者的**相同点**:

回溯法在实现上也是遵循**深度优先**的，即一步一步往前探索，而不像广度优先遍历那样，由近及远一片一片地搜索。

两者的**不同点**:

(1) **访问序不同**: 深度优先遍历目的是“遍历”，本质是无序的。而回溯法目的是“求解过程”，本质是有序的。

(2) **访问次数的不同**: 深度优先遍历对已经访问过的顶点不再访问，所有顶点仅访问一次。而回溯法中已经访问过的顶点可能再次访问。

(3) **剪枝的不同**: 深度优先遍历不含剪枝，而很多回溯算法采用剪枝条件剪除不必要的分枝以提高效能。

回溯法定义

- 狭义定义：

回溯法=**DFS**+剪枝

- 广义定义：

带回退（包括递归）的算法，都是回溯算法

5.1.5 回溯法算法的时间分析

- 通常情况下，回溯法的效率会高于蛮力法。
- 解空间树为子集树时，对应算法的时间复杂度为 $O(2^n)$
- 解空间树为排列树时，对应算法的时间复杂度为 $O(n!)$

5.2 求解0/1背包问题

问题描述：有 n 个重量分别为 $\{w_1, w_2, \dots, w_n\}$ 的物品，它们的价值分别为 $\{v_1, v_2, \dots, v_n\}$ ，给定一个容量为 W 的背包。

设计从这些物品中选取一部分物品放入该背包的方案，**每个物品要么选中要么不选中**，要求选中的物品不仅能够放到背包中，而且重量和恰好为 W 具有最大的价值。

问题求解：第3章采用动态规划法求解，这里采用回溯法求解该问题。

用 $x[1..n]$ 数组存放最优解，其中每个元素取1或0， $x[i]=1$ 表示第 i 个物品放入背包中， $x[i]=0$ 表示第 i 个物品不放入背包中。

为了更清楚地描述算法，将这些给定的算法输入设计成全局变量。这是一个求**最优解**问题。

对第 i 层上的某个分枝结点，对应的状态为 $\text{dfs}(i, \text{tw}, \text{tv}, \text{op})$ ，其中 tw 表示装入背包中的物品总重量， tv 表示背包中物品总价值， op 记录一个解向量。该状态的两种扩展如下：

(1) 选择第 i 个物品放入背包： $\text{op}[i]=1$ ， $\text{tw}=\text{tw}+w[i]$ ， $\text{tv}=\text{tv}+v[i]$ ，转向下一个状态 $\text{dfs}(i+1, \text{tw}, \text{tv}, \text{op})$ 。该决策对应左分枝。

(2) 不选择第 i 个物品放入背包： $\text{op}[i]=0$ ， tw 不变， tv 不变，转向下一个状态 $\text{dfs}(i+1, \text{tw}, \text{tv}, \text{op})$ 。该决策对应右分枝。

叶子结点表示已经对 n 个物品做了决策，对应一个解。对所有叶子结点进行比较求出满足 $\text{tw} \leq W$ 的最大 tv ，用 maxv 表示，对应的最优解 op 存放到 x 中。

0/1背包问题 ($W=6$) :

物品编号	重量	价值
1	5	4
2	3	4
3	2	3
4	1	1

解空间树

物品编号	重量	价值
1	5	4
2	3	4
3	2	3
4	1	1

0/1背包问题 ($W=6$)

x_1 : 选或不选品1

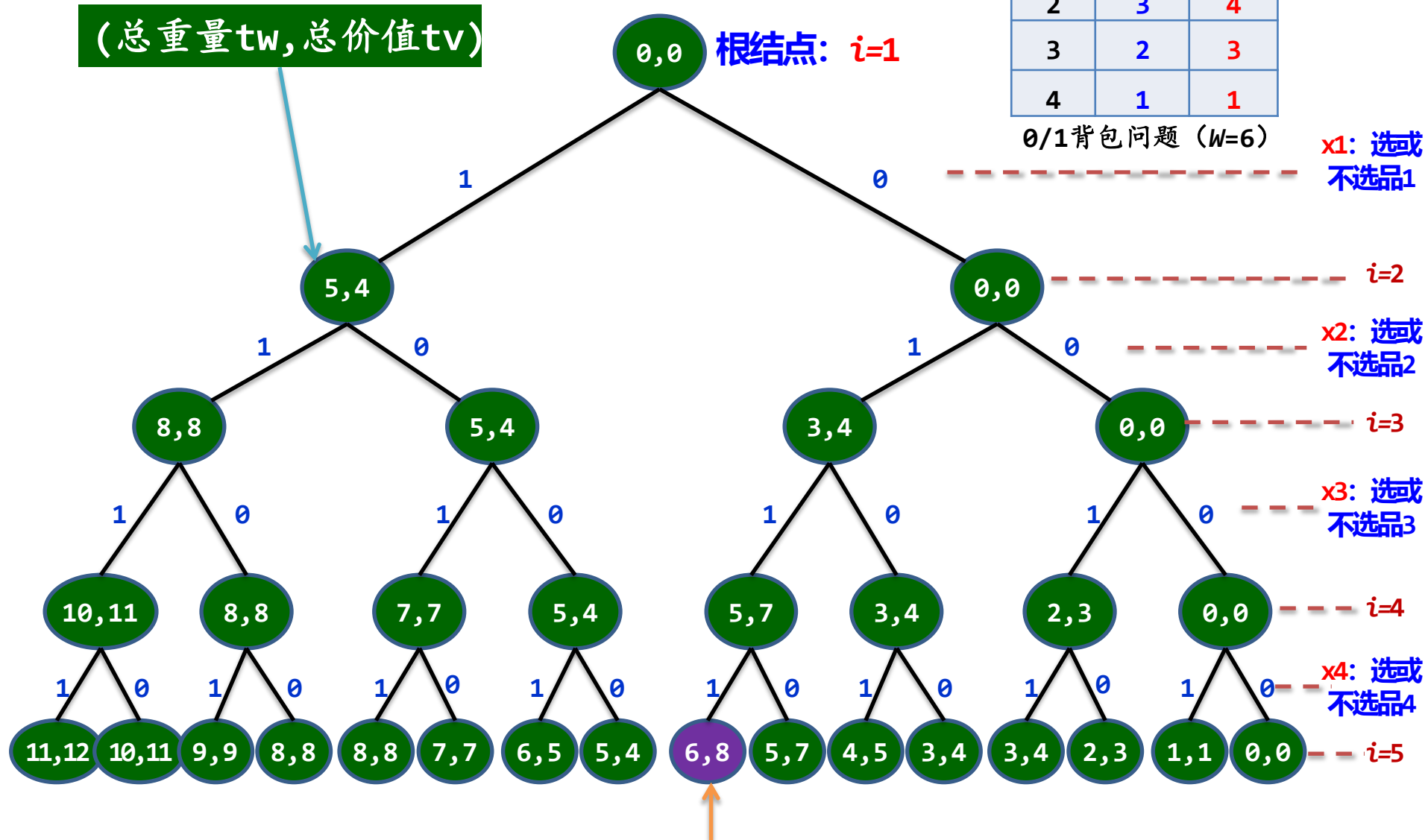
x_2 : 选或不选品2

x_3 : 选或不选品3

x_4 : 选或不选品4

(总重量tw, 总价值tv)

根结点: $i=1$



最优解

//问题表示

int n=4;

int W=6;

int w[]={0,5,3,2,1};

int v[]={0,4,4,3,1};

//求解结果表示

int x[MAXN];

int maxv;

//4种物品

//限制重量为6

//存放4个物品重量,不用下标0元素

//存放4个物品价值,不用下标0元素

//存放最终解

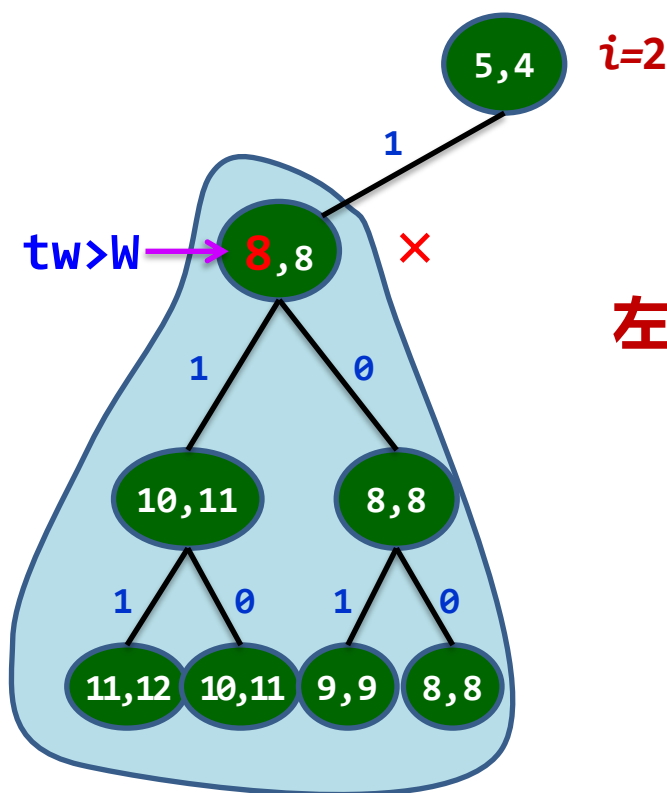
//存放最优解的总价值

采用解空间为子集树递归的算法框架

```
void dfs(int i,int tw,int tv,int op[]) //求解0/1背包问题
{  if (i>n)                               //找到一个叶子结点
    {  if (tw<=W && tv>maxv)               //找到一个满足条件的更优解,保存
        {  maxv=tv;
            for (int j=1;j<=n;j++)
                x[j]=op[j];
        }
    }
    else                                   //尚未找完所有物品
    {  op[i]=1;                             //选取第i个物品
        dfs(i+1,tw+w[i],tv+v[i],op);
        op[i]=0;                           //不选取第i个物品,回溯
        dfs(i+1,tw,tv,op);
    }
}
```

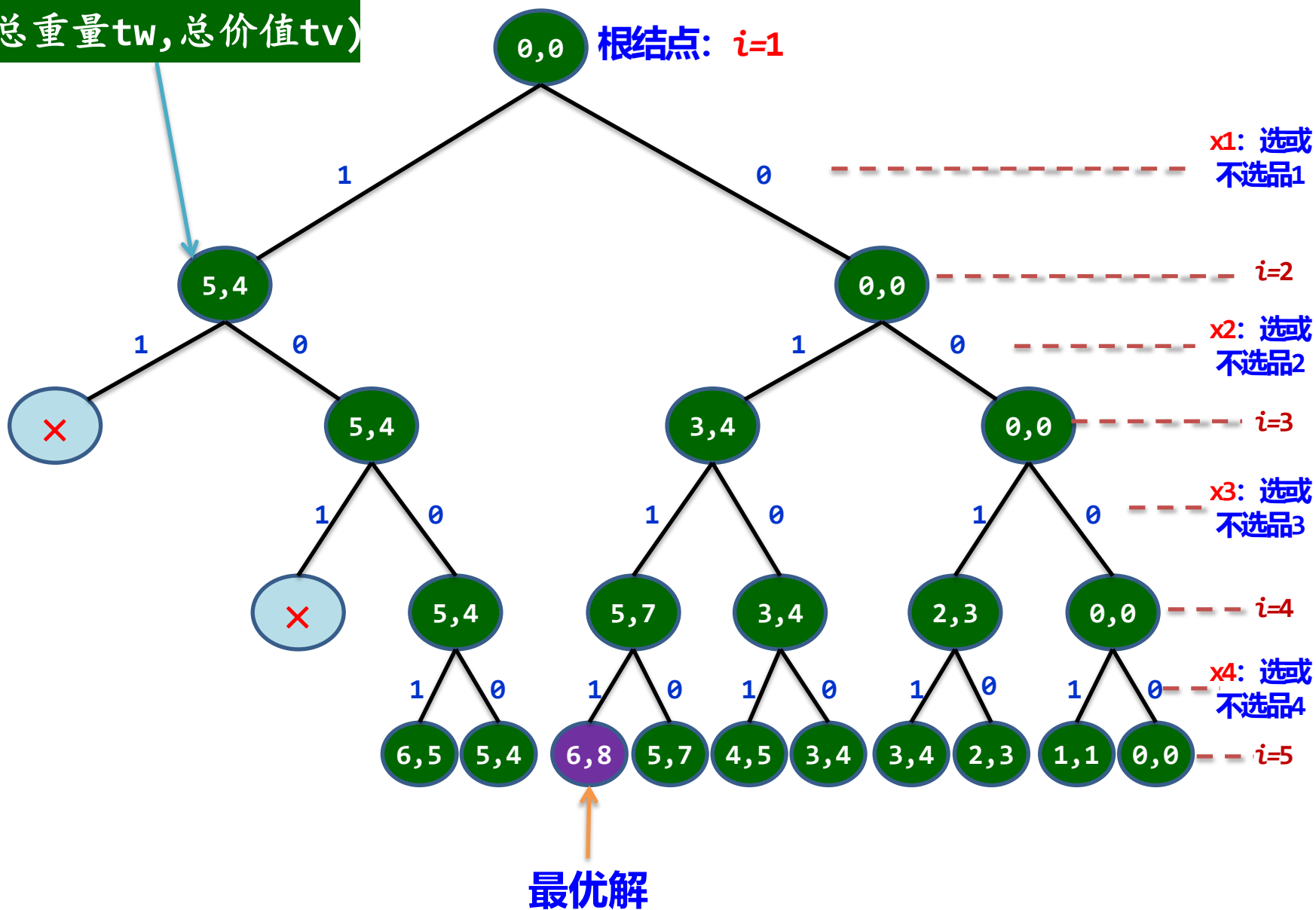
改进：左剪枝

对于第 i 层的有些结点， $tw+w[i]$ 已超过了 W ，显然再选择 $a[i]$ 是不合适的。如第2层的 $(5, 4)$ 结点， $tw=5$ ， $w[2]=3$ ，而 $tw+w[2]>W$ ，选择物品2进行扩展是不必要的，可以增加一个限界条件进行剪枝，如若选择物品 i 会导致超重即 $tw+w[i]>W$ ，就不再扩展该结点，也就是仅仅扩展 $tw+w[i]\leq W$ 的左孩子结点。



左剪枝条件: $tw+w[i] < W$

(总重量tw, 总价值tv)




```

void dfs(int i,int tw,int tv,int op[]) //求解0/1背包问题
{
    if (i>n) //找到一个叶子结点
    {
        if (tw<=W && tv>maxv) //找到一个满足条件的更优解,保存
        {
            maxv=tv;
            maxw=tw;
            for (int j=1;j<=n;j++)
                x[j]=op[j];
        }
    }
    else //尚未找完所有物品
    {
        if (tw+w[i] <= W) //左孩子结点剪枝
        {
            op[i]=1; //选取第i个物品
            dfs(i+1,tw+w[i],tv+v[i],op);
        }
        op[i]=0; //不选取第i个物品,回溯
        dfs(i+1,tw,tv,op);
    }
}

```

改进：右剪枝

- 用 rw 表示考虑第 i 个物品时剩余物品的重量。

即 $rw=w[i]+\dots+w[n]$, 初始时, rw 是所有物品的和

- 当不选择物品 i 时, 若 $tw+rw \leq W$ (注意 rw 中包含 $w[i]$) 时, 也就是说即使选择后面的所有物品, 重量也不会达到 W , 因此不必要再考虑扩展这样的结点
- 也就是说, 对于右分枝仅仅扩展 $tw+rw > W$ 的结点。

物品编号	重量	价值
1	5	4
2	3	4
3	2	3
4	1	1

0/1背包问题 ($W=6$)

x_1 : 选或不选品1

x_2 : 选或不选品2

$i=3$

x_3 : 选或不选品3

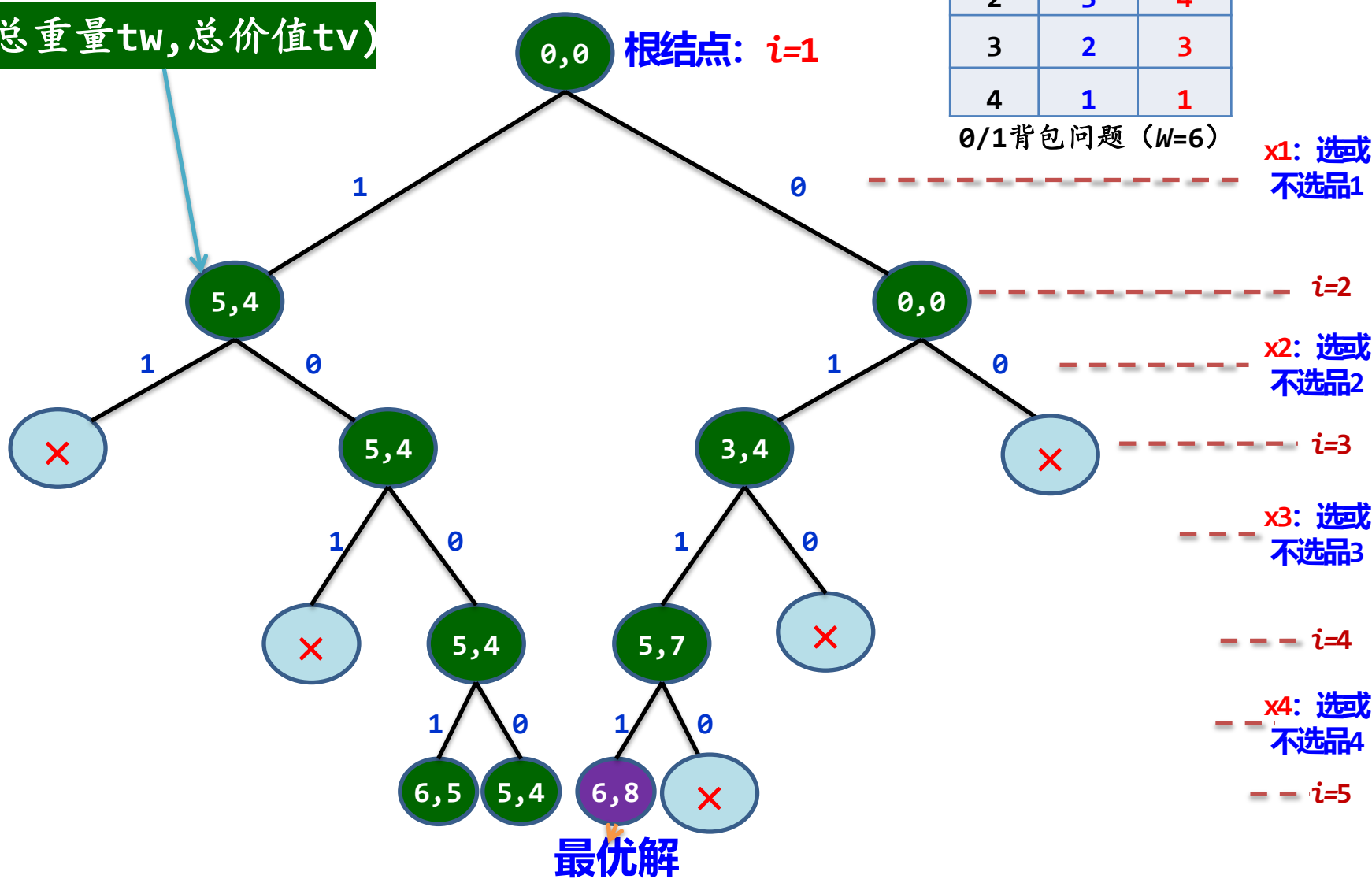
$i=4$

x_4 : 选或不选品4

$i=5$

(总重量 tw , 总价值 tv)

根结点: $i=1$



```

void dfs(int i,int tw,int tv,int rw,int op[]) //求解0/1背包问题
{ //初始调用时rw为所有物品重量和
    int j;
    if (i>n) //找到一个叶子结点
    { if (tw==W && tv>maxv) //找到一个满足条件的更优解,保存
      { maxv=tv;
        for (j=1;j<=n;j++) //复制最优解
          x[j]=op[j];
      }
    }
    else //尚未找完所有物品
    { if (tw+w[i]<=W) //左孩子结点剪枝
      { op[i]=1; //选取第i个物品
        dfs(i+1,tw+w[i],tv+v[i],rw-w[i],op);
      }
      op[i]=0; //不选取第i个物品,回溯
      if (tw+rw>W) //右孩子结点剪枝
        dfs(i+1,tw,tv,rw-w[i],op);
    }
}
}

```

【算法分析】 该算法不考虑剪枝时解空间树中有 $2^{n+1}-1$ 个结点，对应的算法时间复杂度为 $O(2^n)$ 。

实验5-1: 01背包问题(回溯法)

- 问题描述: 有 n 个物品, 每个物品有重量 w 和价值 v 两个属性, 还有一个承重为 W 的背包。现在让你用这个背包装物品, 使所装物品的价值最大化。

- 示例 1:

输入: $w = 6$,

输出:

items	A: 0	B: 1	C: 2	D: 3
profit	1	6	10	16
weight	1	2	3	5

要求:

- 1 填空完成程序;
- 2 输出所选物品重量及价值, 总重量与价值。
- *3 报告中指出是排列树还是子集树, 指出约束函数与限界函数。

5.3 求解装载问题

5.3.1 求解简单装载问题

问题描述: 有 n 个集装箱要装上一艘载重量为 W 的轮船，其中集装箱 i ($1 \leq i \leq n$) 的重量为 w_i 。不考虑集装箱的体积限制，现要这些集装箱中选出若干装上轮船，使它们的重量之和等于 W ，当总重量相同时要求选取的集装箱个数尽可能少。 **例如:** $n=5$, $W=10$, $w=\{5, 2, 6, 4, 3\}$ 时，其最佳装载方案是 $(0, 0, 1, 1, 0)$ ，即装载第3、4个集装箱。

问题求解: 采用带剪枝的回溯法求解。问题的表示如下：

```
int w[]={0, 5, 2, 6, 4, 3}; //各集装箱重量，不用下标0的元素
int n=5, W=10;
```

求解的结果表示如下：全局变量

```
int maxw;                //存放最优解的总重量
int x[MAXN];             //存放最优解向量
int minnum=999999;       //存放最优解的集装箱个数，初值为最大值
void dfs(int num, int tw, int rw, int op[], int i)
```

其中 num 表示选择的集装箱个数， tw 表示选择的集装箱重量和， rw 表示剩余集装箱的重量和， op 表示一个解，即一个选择方案， i 表示考虑的集装箱 i 。

```

void dfs(int num,int tw,int rw,int op[],int i) //考虑第i个集装箱
{
    if (i>n) //找到一个叶子结点
    {
        if (tw==W && num<minnum)
        {
            maxw=tw; //找到一个满足条件的更优解,保存它
            minnum=num;
            for (int j=1;j<=n;j++)
                x[j]=op[j]; //复制最优解
        }
    }
    else //尚未找完所有集装箱
    {
        op[i]=1; //选取第i个集装箱
        if (tw+w[i]<=W) //左孩子结点剪枝: 装载满足条件的集装箱
            dfs(num+1,tw+w[i],rw-w[i],op,i+1);
        op[i]=0; //不选取第i个集装箱,回溯
        if (tw+rw>W) //右孩子结点剪枝
            dfs(num,tw,rw-w[i],op,i+1);
    }
}

```

```
int w[]={0, 5, 2, 6, 4, 3};    //各集装箱重量，不用下标0的元素  
int n=5, W=10;
```



最优方案
选取第3个集装箱
选取第4个集装箱
总重量=10

5.3.2 求解复杂装载问题

问题描述: 有一批共 n 个集装箱要装上**两艘**载重量分别为 c_1 和 c_2 的轮船，其中集装箱 i 的重量为 w_i ，且 $w_1+w_2+\dots+w_n \leq c_1+c_2$ 。

装载问题要求确定是否有一个合理的装载方案可将这些集装箱装上这两艘轮船。如果有，找出一种装载方案。

例如，当 $n=3$ ， $c_1=c_2=50$ ， $w=\{10, 40, 40\}$ 时，则可以将集装箱1和2装到第一艘轮船上，而将集装箱3装到第二艘轮船上。如果 $w=\{20, 40, 40\}$ ，则无法将这3个集装箱都装上轮船。

问题求解: 如果一个给定的复杂装载问题有解，则可以采用如下方式得到一个装载方案：

首先将第一艘轮船尽可能装满，然后将剩余的集装箱装在第二艘轮船上。

可以用反证法证明其正确性。

```

void dfs(int tw,int rw,int op[],int i) //求第一艘轮船的最优解
{
    if (i>n) //找到一个叶子结点
    {
        if (tw<=c1 && tw>maxw)
        {
            maxw=tw; //找到一个满足条件的更优解
            for (int j=1;j<=n;j++) //复制最优解
                x[j]=op[j];
        }
    }
    else //尚未找完所有集装箱
    {
        op[i]=1; //选取第i个集装箱
        if (tw+w[i]<=c1) //左孩子结点剪枝
            dfs(tw+w[i],rw-w[i],op,i+1);

        op[i]=0; //不选取第i个集装箱,回溯
        if (tw+rw>c1) //右孩子结点剪枝
            dfs(tw,rw-w[i],op,i+1);
    }
}

```

```
bool solve()  
{  int sum=0;  
    for (int j=1;j<=n;j++)  
        if (x[j]==0)  
            sum+=w[j];  
    if (sum<=c2)  
        return true;  
    else  
        return false;  
}
```

//求解复杂装载问题

//累计第一艘轮船装完后剩余的集装箱重量

//第二艘轮船可以装完

//第二艘轮船不能装完

```

void main()
{
    int op[MAXN];                //存放临时解
    memset(op,0,sizeof(op));
    int rw=0;
    for (int i=1;i<=n;i++)
        rw+=w[i];

    dfs(0,rw,op,1);              //求第一艘轮船的最优解
    printf("求解结果\n");

    if (solve())                  //输出结果
    {
        printf("    装载方案\n");
        dispasolution(n);
    }
    else printf("    没有合适的装载方案\n");
}

```

//问题表示

```
int w[]={0,10,40,40}; //各集装箱重量,不用下标0的元素
```

```
int n=3;
```

```
int c1=50,c2=50;
```



求解结果

装载方案

将第1个集装箱装上第一艘轮船

将第2个集装箱装上第一艘轮船

将第3个集装箱装上第二艘轮船

实验5-2: 装载问题(回溯法)

- 问题描述: 有一批共 n 个集装箱要装上两艘载重量分别为 c_1 和 c_2 的轮船, 其中集装箱 i 的重量为 w_i , 且 $w_1 + w_2 + \dots + w_n \leq c_1 + c_2$ 。装载问题要求确定是否有一个合理的装载方案可将这些集装箱装上这两艘轮船。如果有, 找出一种装载方案。

- 示例 1:

当 $n=3$, $c_1=c_2=50$, $w=\{10, 40, 40\}$ 时, 则可以将集装箱1和2装到第一艘轮船上, 而将集装箱3装到第二艘轮船上。如果 $w=\{20, 40, 40\}$, 则无法将这3个集装箱都装上轮船。

- 要求:

- 1 填空完成程序;
- 2 输出装载方案。

***3** 报告中指出是排列树还是子集树, 指出约束函数与限界函数。

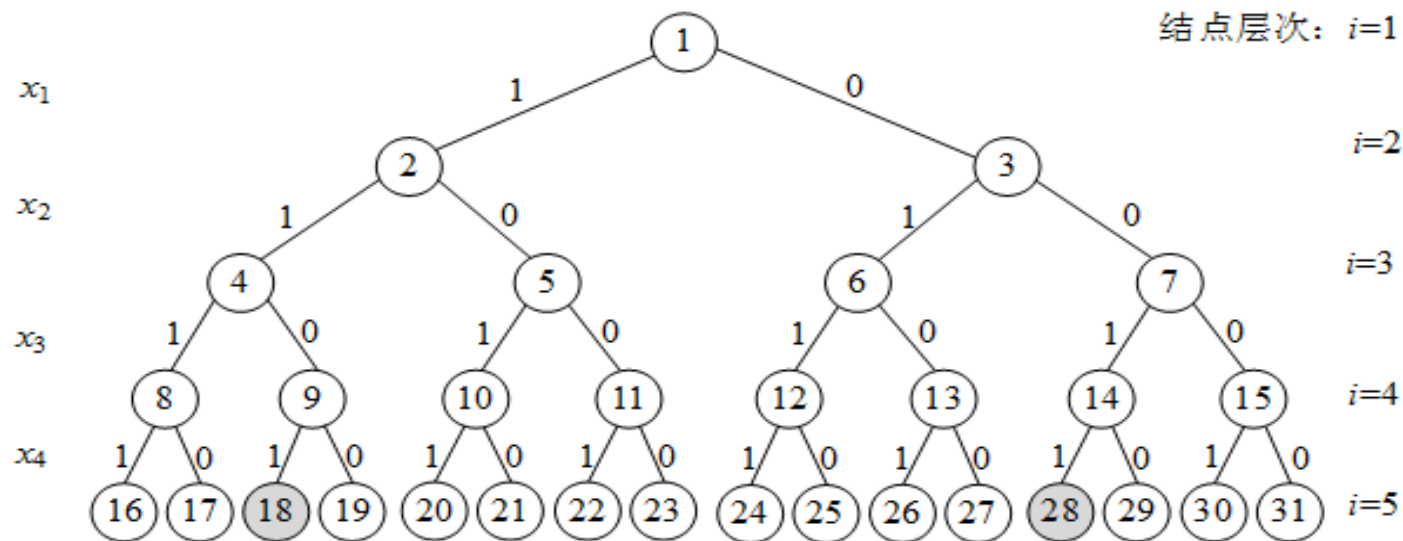
5.4 求解子集和问题

5.4.1 求子集和问题的解

问题描述: 给定 n 个不同的正整数集合 $w = (w_1, w_2, \dots, w_n)$ 和一个正数 W , 要求找出 w 的子集 s , 使该子集中所有元素的和为 W 。

例如, 当 $n=4$ 时, $w = (11, 13, 24, 7)$, $W=31$, 则满足要求的子集为 $(11, 13, 7)$ 和 $(24, 7)$ 。

问题求解: 当 $n=4$ 时的解空间树如下:



从 i 层到 $i+1$ 层 ($1 \leq i \leq n$) 的每一条边标有 x_i 的值, x_i 或者为1或者为0, x_i 为1时表示取 w_i 整数, x_i 为0时表示不取 w_i 整数, 从根结点到叶子结点的所有路径定义了解空间。

用 tw 表示选取的整数和， rw 表示余下的整数和，设置相关的剪枝函数如下：

- **约束函数：**通过检查当前整数 $w[i]$ 加入后子集和是否超过 W ，若是则不能选择该路径。这用于左孩子结点的剪枝。
- **限界函数：**如果一个结点满足 $tw+rw < W$ ，也就是说即便选择剩余所有整数，也不可能找到一个解。这用于右孩子结点的剪枝。


```

void dfs(int tw,int rw,int x[],int i) //求解子集和
{ //tw为考虑第i个整数时选取的整数和, rw为剩下的整数和
    if (i>n) //找到一个叶子结点
    { if (tw==W) //输出一个满足条件的解
        dispasolution(x);
    }

    else //尚未找完所有整数
    { if (tw+w[i]<=W) //左孩子结点剪枝: 选取满足条件的整数w[i]
        { x[i]=1; //选取第i个整数
            dfs(tw+w[i],rw-w[i],x,i+1);
        }

        if (tw+rw>W) //右孩子结点剪枝: 剪除不可能存在解的结点
        { x[i]=0; //不选取第i个整数,回溯
            dfs(tw,rw-w[i],x,i+1);
        }
    }
}

```

```
int n=4,W=31;  
int w[]={0,11,13,24,7};
```

//存放所有整数,不用下标0的元素



第1个解:
选取的数为11 13 7
第2个解:
选取的数为24 7

算法分析: 算法的解空间树中有 $2^{n+1}-1$ 个结点, 对应的算法时间复杂度为 $O(2^n)$ 。

5.4.2 判断子集和问题是否存在解

采用回溯法一般是针对问题存在解时求出相应的一个或多个解，或者最优解。

如果要判断问题是否存在解（一个或者多个），可以将求解函数改为**bool**类型，当找到任何一个解时返回**true**，否则返回**false**。需要注意的是当问题没有解时需要搜索所有解空间。

```

bool dfs(int tw,int rw,int i)           //求解子集和
{   if (i>n)                            //找到一个叶子结点
    {   if (tw==W)                       //找到一个满足条件的解,输出
        return true;
    }

    else                                //尚未找完所有物品
    {   if (tw+w[i]<=W)                  //左孩子结点剪枝
        return dfs(tw+w[i],rw-w[i],i+1); //选取第i个整数

        if (tw+rw>W)                    //右孩子结点剪枝
            return dfs(tw,rw-w[i],i+1); //不选取第i个整数,回溯
    }
    return false;
}

```

另外一种方法是通过解个数来判断，如设置全局变量count表示解个数，初始化为0，调用搜索解的回溯算法，当找到一个解时置count++。最后判断count>0算法成立，若为真，表示存在解，否则表示不存在解。

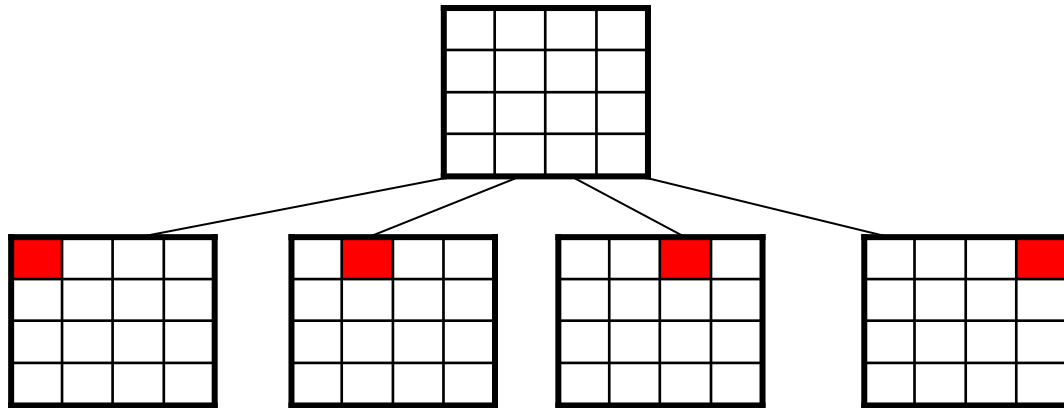
5.5 n后问题

八皇后问题是十九世纪著名的数学家高斯于1850年提出的。问题是：在 8×8 的棋盘上摆放八个皇后，使其不能互相攻击，即任意两个皇后都不能处于同一行、同一列或同一斜线上。可以把八皇后问题扩展到**n皇后问题**，即在 $n \times n$ 的棋盘上摆放n个皇后，使任意两个皇后都不能处于**同一行、同一列或同一斜线上**。

1				Q				
2						Q		
3								Q
4		Q						
5							Q	
6	Q							
7			Q					
8					Q			
	1	2	3	4	5	6	7	8

5.5 n后问题

- **确定问题状态：**问题的状态即棋盘的布局状态。
- **构造状态空间树：**状态空间树的根为空棋盘，每个布局的下一步可能布局是该布局结点的子结点。
 - 由于可以预知，在每行中有且只有一个皇后，因此可采用逐行布局的方式，即每个布局有 n 个子结点。

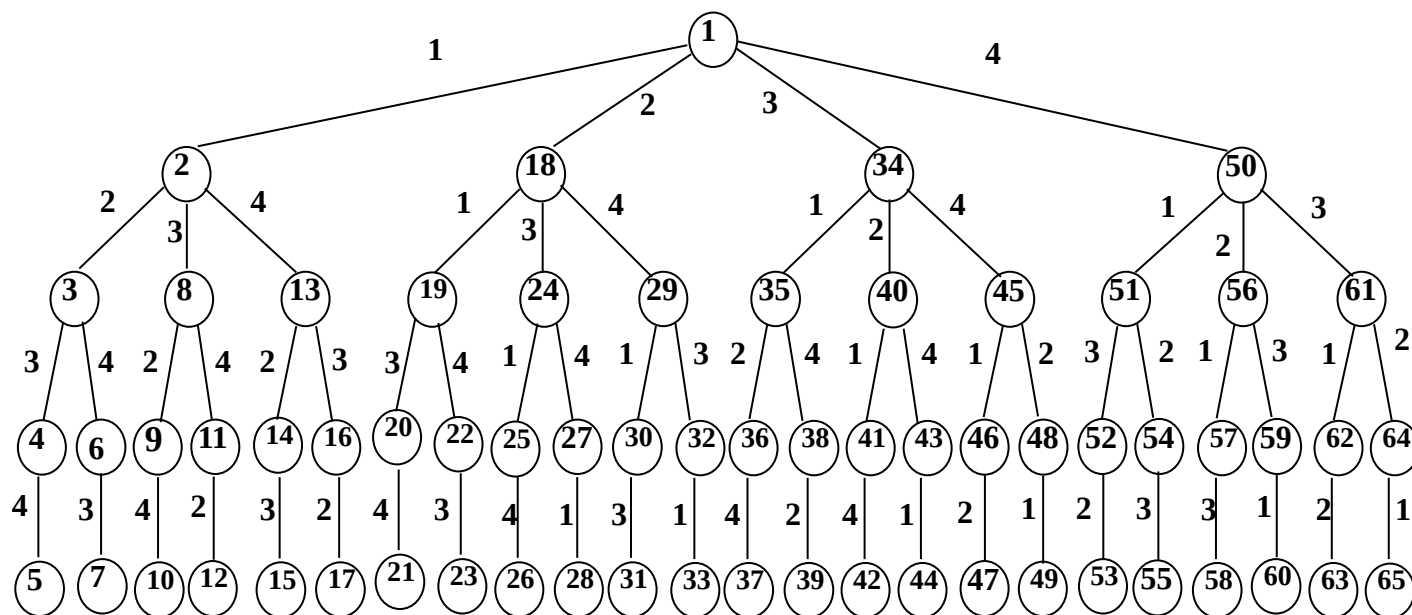


5.5 n后问题

- 设4个皇后为 x_i , 分别在第 i 行($i=1, 2, 3, 4$);
- 问题的解状态: 可以用 $(1, x_1), (2, x_2), \dots, (4, x_4)$ 表示4个皇后的位置;
- 由于行号固定, 可简单记为: (x_1, x_2, x_3, x_4) ; 例如: $(4, 2, 1, 3)$
- 问题的解空间: $(x_1, x_2, x_3, x_4), 1 \leq x_i \leq 4 (i=1, 2, 3, 4)$, 共 $4!$ 个状态;

5.5 n后问题

4皇后问题解空间的树结构



5.5 n后问题

约束条件

- 任意两个皇后不能位于同一行上;
- 任意两个皇后不能位于同一列上, 所以解向量X必须满足约束条件:

$$\text{当 } i \neq j \text{ 时, } x_i \neq x_j \quad (\text{式1})$$

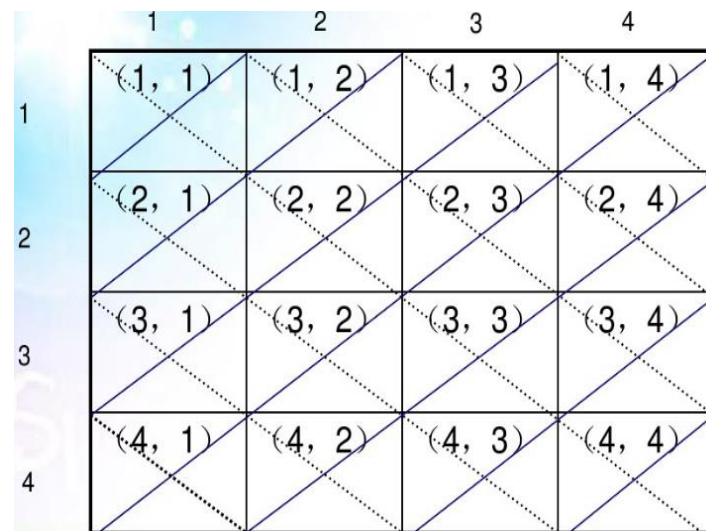
1		Q		
2				Q
3	Q			
4			Q	
	1	2	3	4

5.5 n后问题

约束条件

- 任意两个皇后不能在一条斜线上：若两个皇后摆放的位置分别是 (i, x_i) 和 (j, x_j) ，若在斜率为-1的斜线上，则有 $i - x_i = j - x_j$ ；若在上斜率为1的斜线上，则有 $i + x_i = j + x_j$ 。综合两种情况，解向量 x 必须满足约束条件：

$$|i - j| \neq |x_i - x_j| \quad (\text{式2})$$



原问题即：在解空间中**寻找符合约束条件**的解状态。

5.5 n 后问题

搜索解空间，剪枝

- (1) 从空棋盘起，逐行放置棋子。
- (2) 每在一个布局中放下一个棋子，即推演到一个新的布局。
- (3) 如果当前行上没有可合法放置棋子的位置，则回溯到上一行，重新布放上一行的棋子。

5.5 n后问题

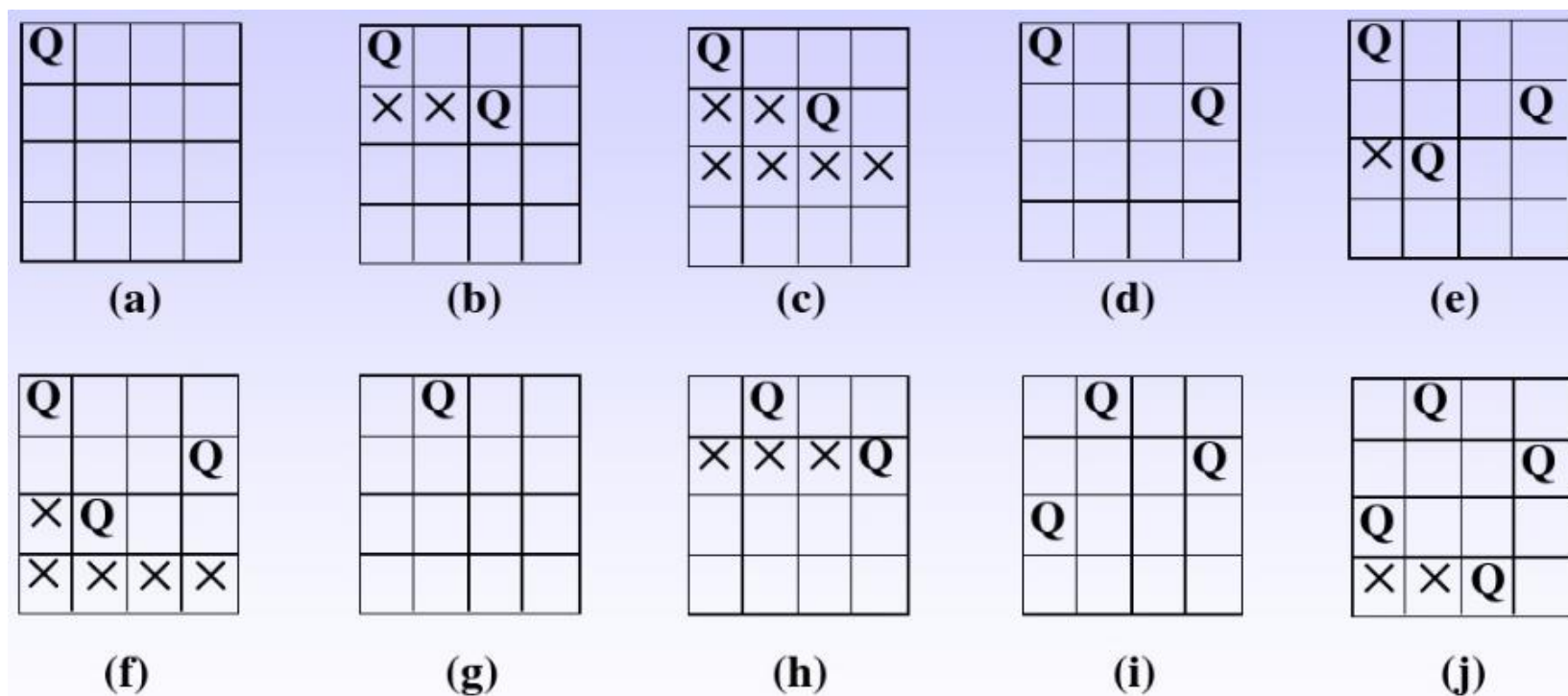
为了简化问题，下面讨论4皇后问题。

4皇后问题的解空间树是一个完全4叉树，树的根结点表示搜索的初始状态，从根结点到第2层结点对应皇后1在棋盘中第1行可能摆放的位置，从第2层到第3层结点对应皇后2在棋盘中第2行的可能摆放的位置，以此类推。

	1	2	3	4	
1					← 皇后1
2					← 皇后2
3					← 皇后3
4					← 皇后4

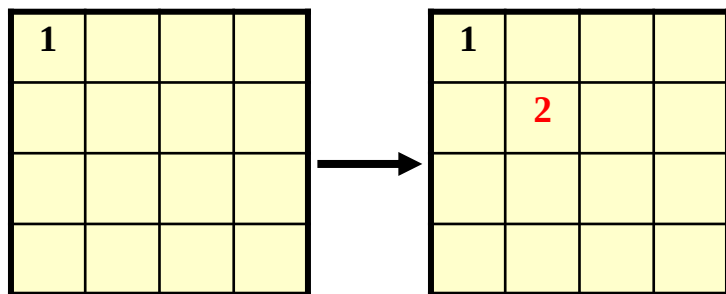
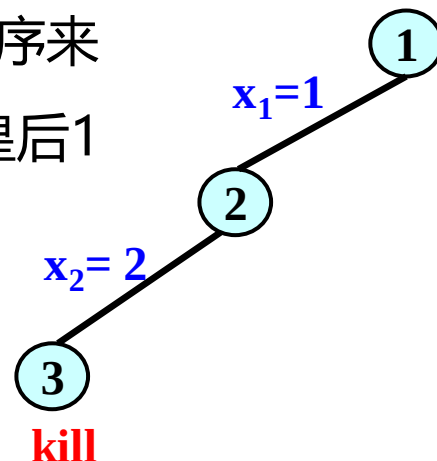
5.5 n后问题

回溯法求解4皇后问题的搜索过程：



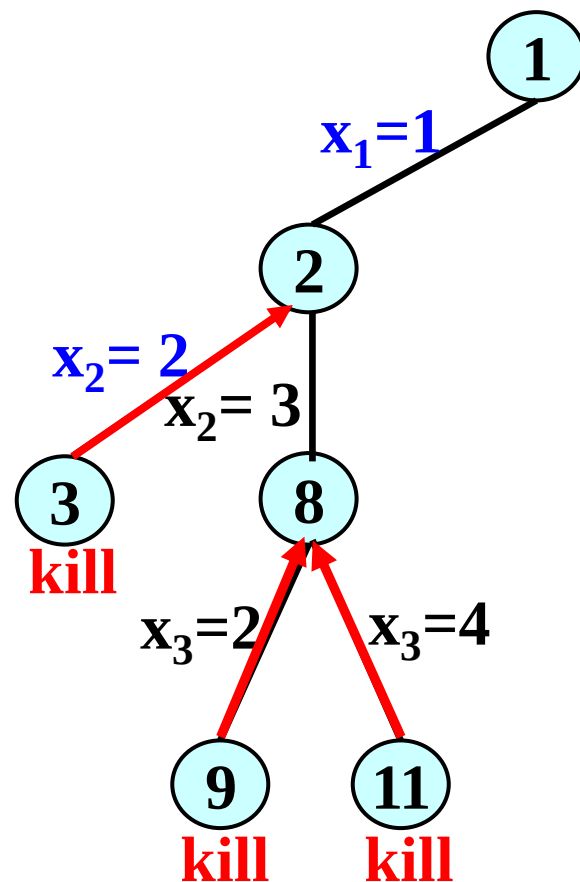
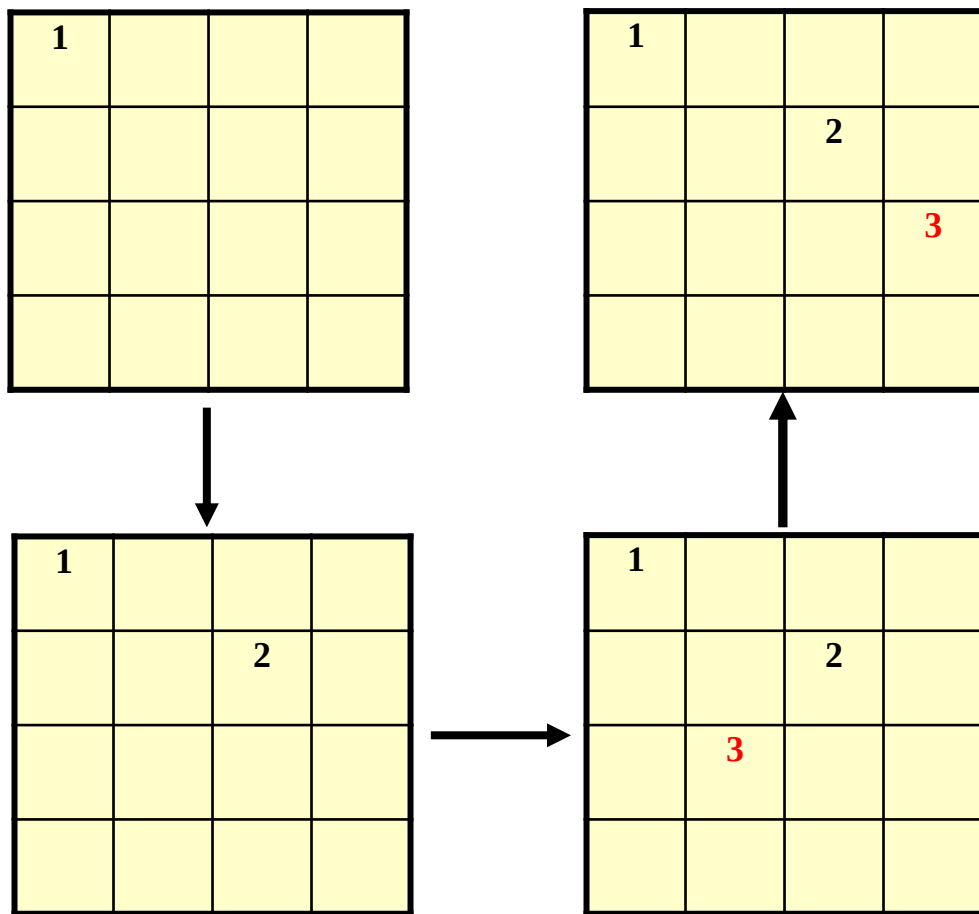
5.5 n后问题

- 开始把根结点1作为唯一的活结点, 根结点就成为E-结点而且路径为(); 接着生成子结点, 假设按自然数递增的次序来生成子结点, 那么结点2被生成, 这条路径为(1), 即把皇后1放在第1列上。
- 结点2变成E-结点, 它再生成结点3, 路径变为(1, 2), 即皇后1在第1列上, 皇后2在第2列上, 所以结点3被杀死, 此时应回溯。



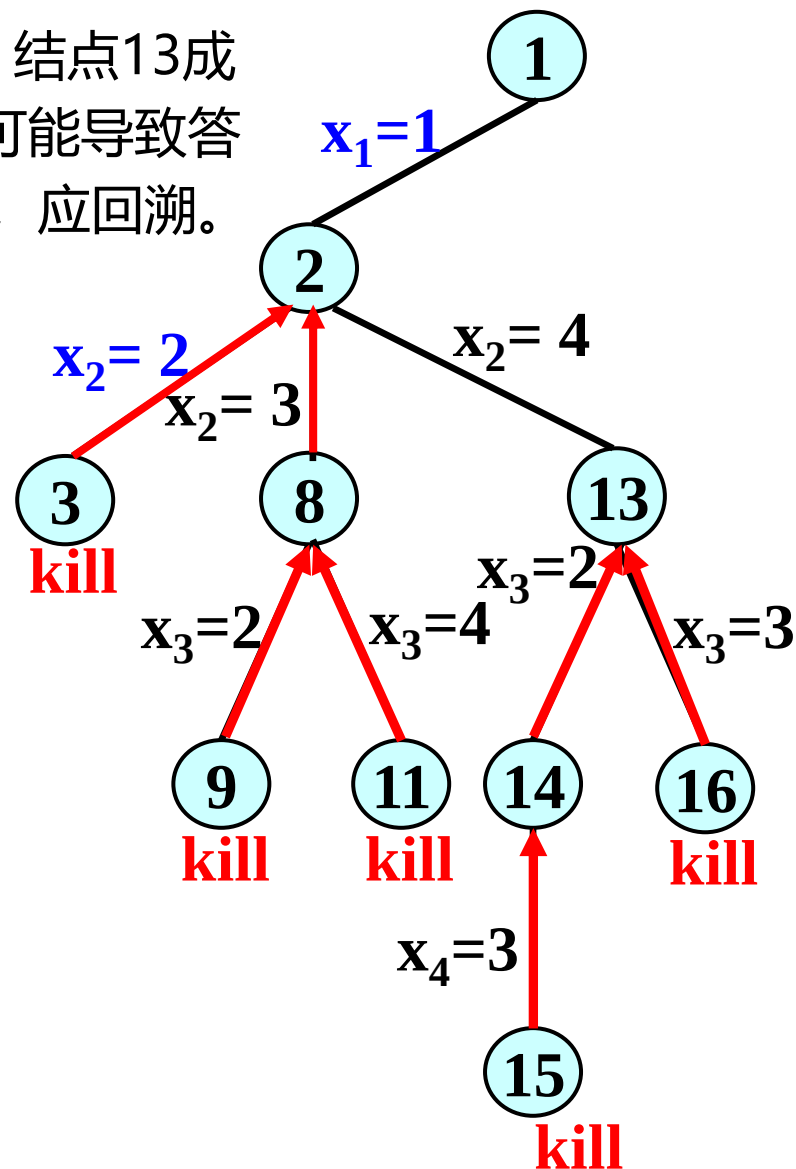
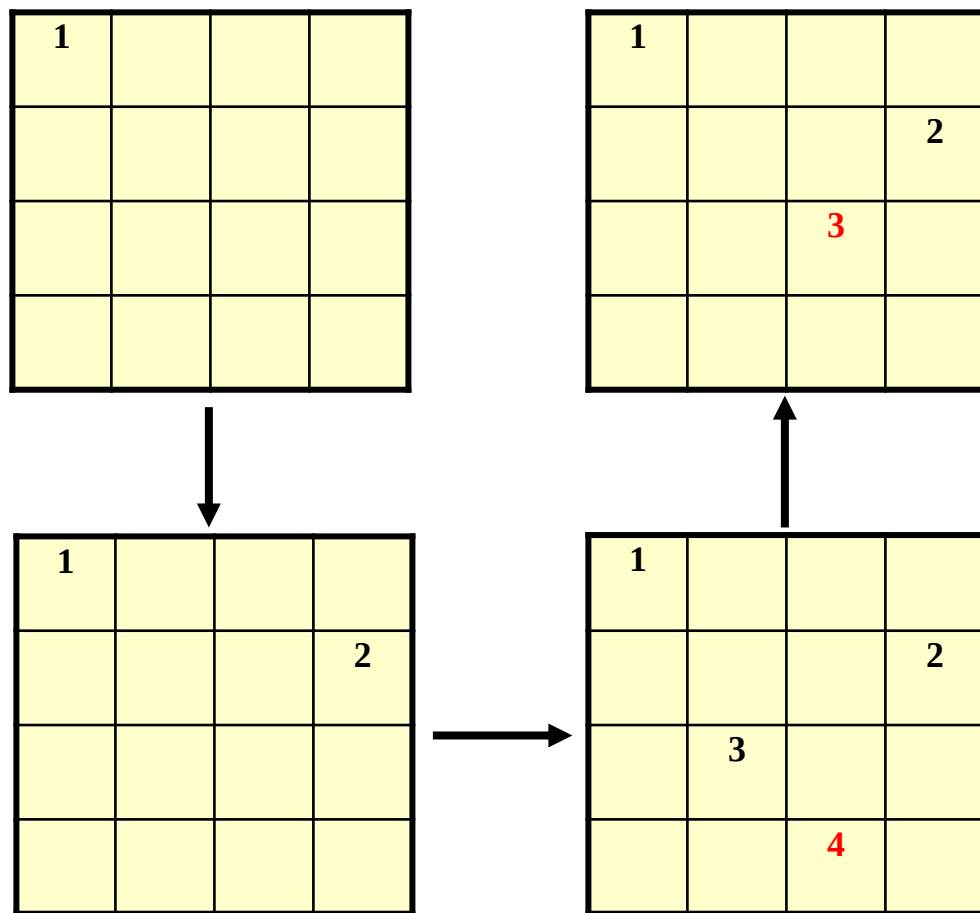
5.5 n后问题

- 回溯到结点2生成结点8, 路径变为(1, 3), 则结点8成为E-结点, 它生成结点9和结点11都会被杀死(即它的儿子表示不可能导致答案的棋盘格局), 所以结点8也被杀死, 应回溯。



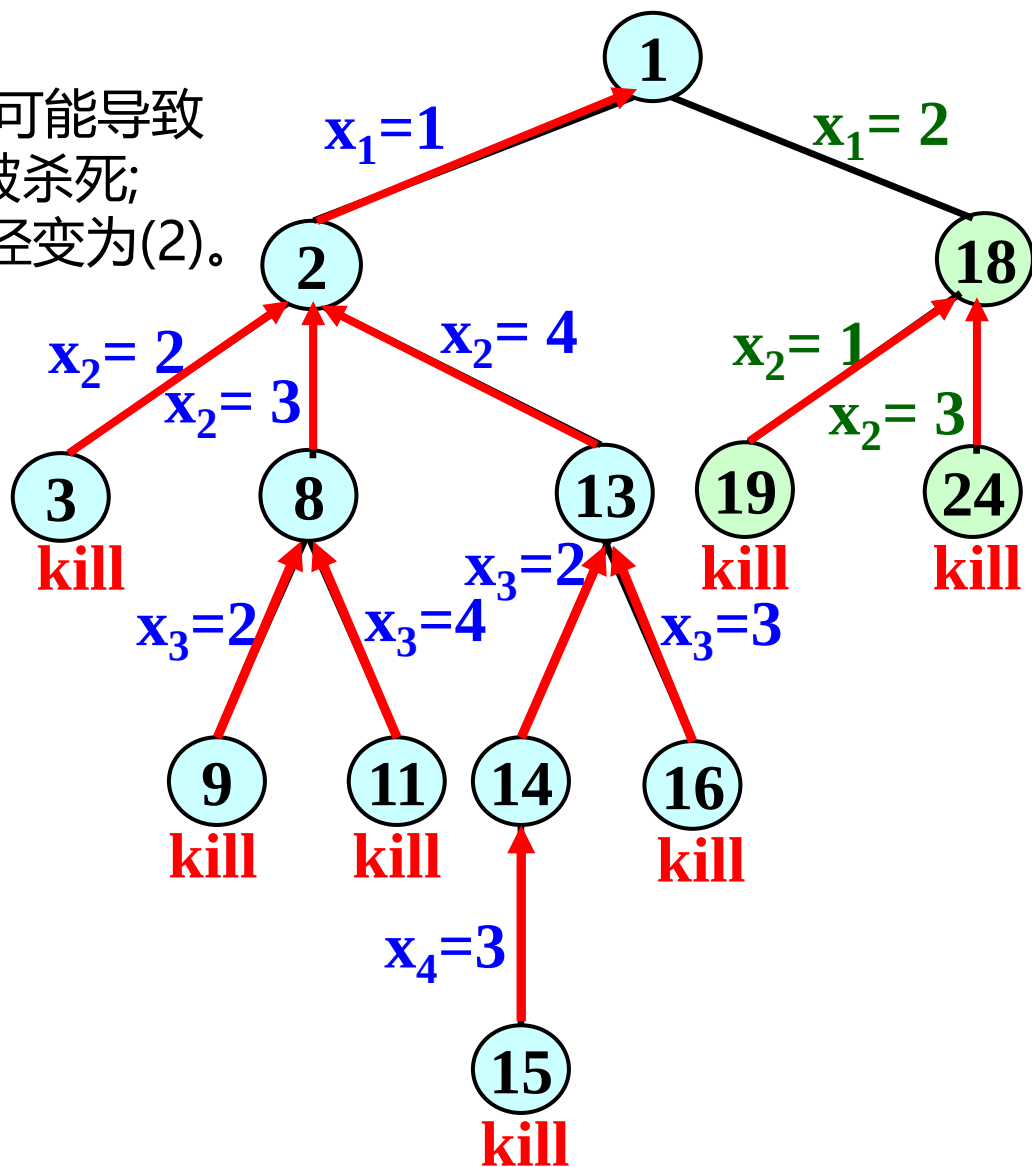
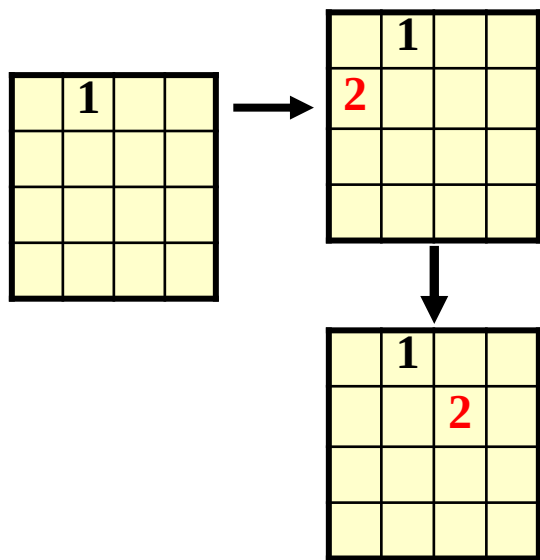
5.5 n后问题

- 回溯到结点2生成结点13, 路径变为(1, 4), 结点13成为E-结点, 由于它的儿子表示的是一些不可能导致答案结点的棋盘格局, 因此结点13也被杀死, 应回溯。



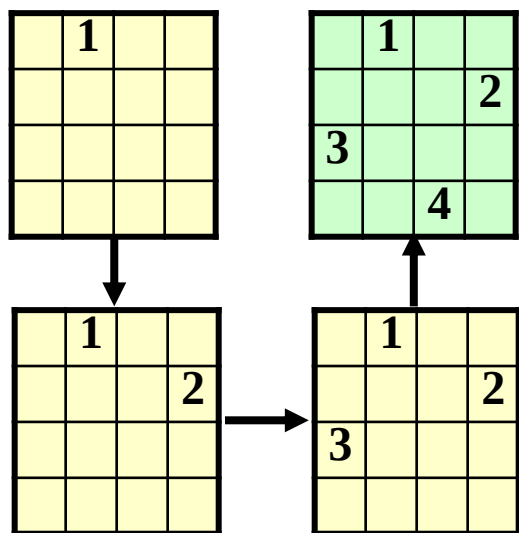
5.5 n后问题

- 结点2的所有儿子表示的都是不可能导致答案的棋盘格局, 因此结点2也被杀死; 再回溯到结点1生成结点18, 路径变为(2)。
- 结点18的子结点19、结点24被杀死, 应回溯。

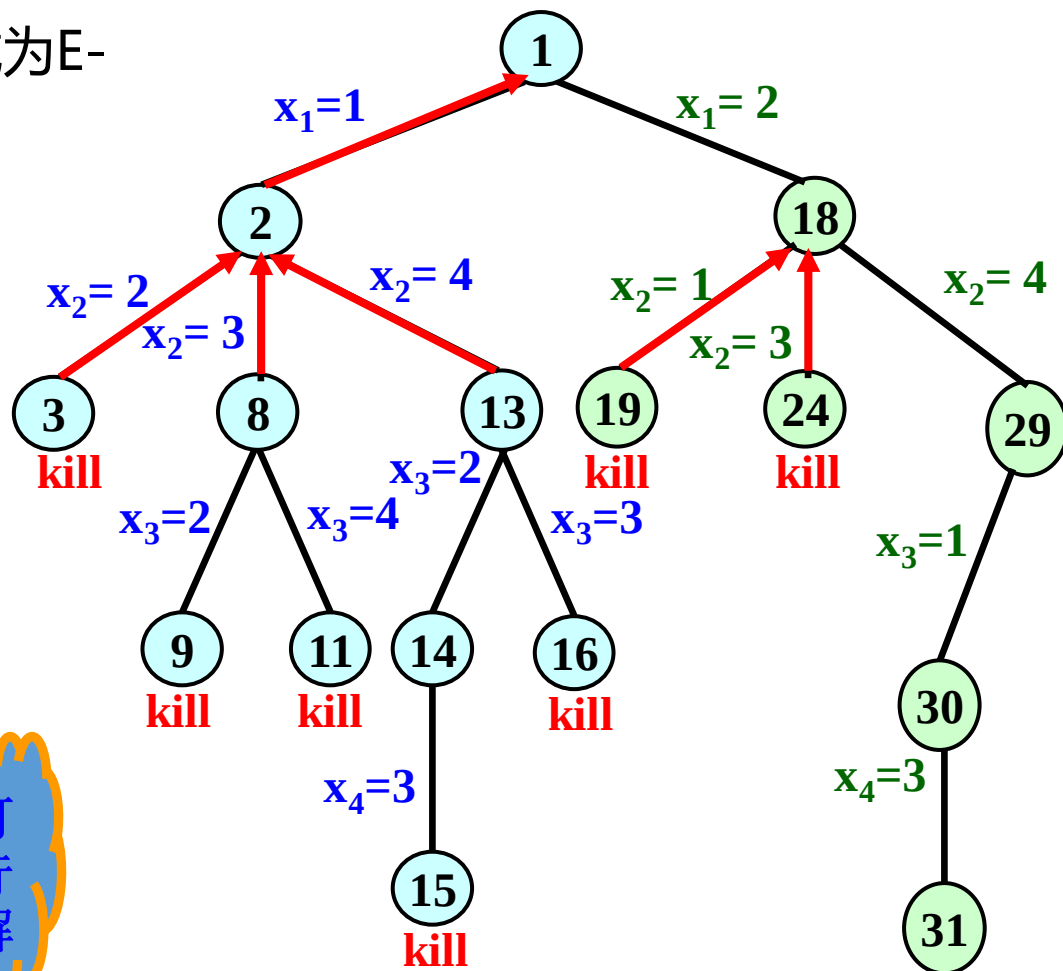


5.5 n后问题

- 结点18生成结点29, 结点29成为E-结点, 路径变为(2,4);
- 结点29生成结点30, 路径变为(2,4,1)
- 结点30生成结点31, 路径变为(2,4,1,3), 找到一个4-王后问题的可行解

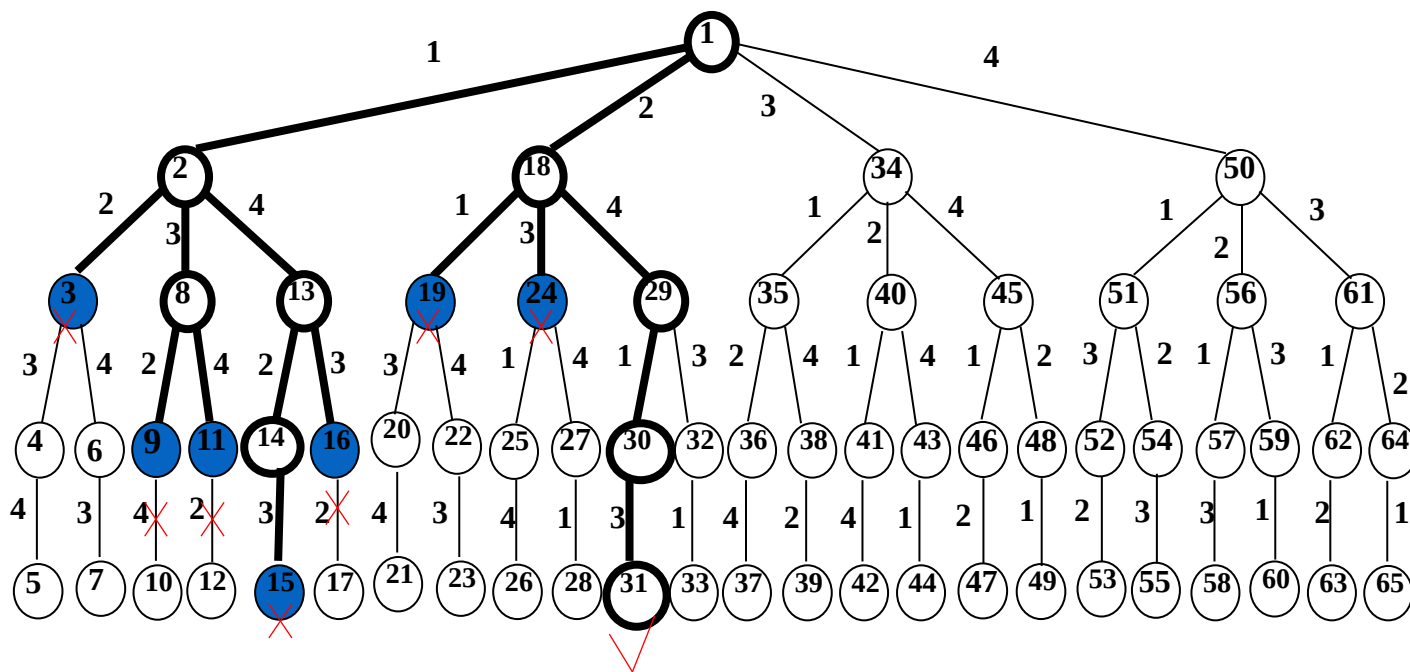


可行解



5.5 n后问题

n=4的n皇后问题的搜索、剪枝与回溯



递归回溯算法 (排列树法):

```
void Queens(vector<int>& q, int i, int length)
{
    if (i == length) //找到一个解
    {
        for (int j = 0; j < length; j++)
        {
            cout << q[j] << " ";
        }
        cout << endl;
    }
    else
    {
        for (int k = i; k < length; k++)
        {
            swap(q, k, i);
            if (place(q, i)) //是否可放置
                Queens(q, i + 1, length);
            swap(q, k, i); //回溯还原
        }
    }
}
```

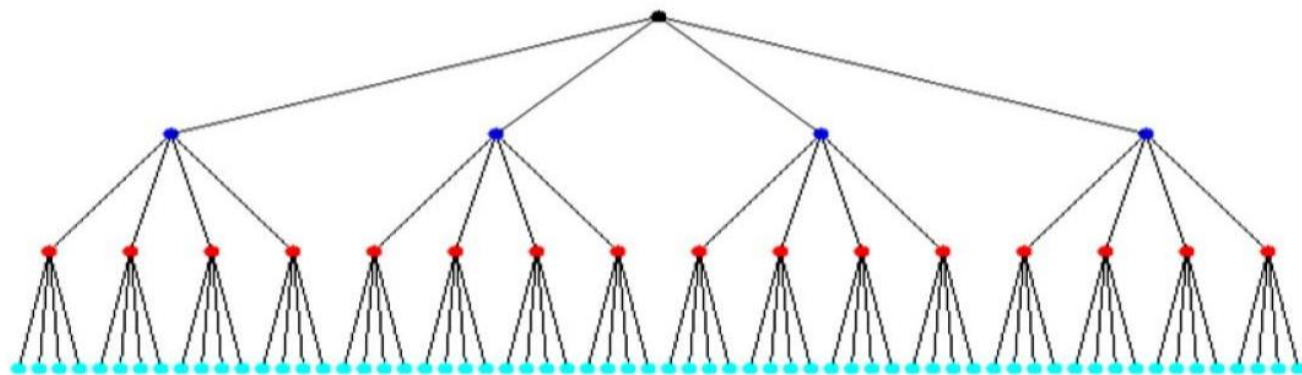
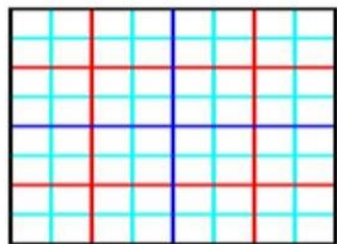
//测试第*i*行的q[i]列上能否摆放皇后

```
bool place(vector<int>& q, int i)
{
    for (int j = 0; j < i; j++)
    {
        //同行或同列或在同一斜线上
        if (i == j || q[i] == q[j] || abs(i - j) == abs(q[i] - q[j]))
        {
            return false;
        }
    }
    return true;
}
```

以n=4皇后问题为例

第二种：解空间是4叉树

每个皇后在一行上有四个可选位置（**显式约束**：任两皇后不同行）



共有 $4^4 = 256$ 种情况

非递归回溯算法对应的算法（子集树法）：

```
void Queens(int n)    //求解n皇后问题
{  int i=1;           //i表示当前行,也表示放置第i个皇后
  q[i]=0;             //q[i]是当前列,每个新考虑的皇后初始位置置为0列
  while (i>=1)        //尚未回溯到头,循环
  {  q[i]++;          //原位置后移动一列
    while (q[i]<=n && !place(i)) //试探一个位置(i,q[i])
      q[i]++;

    if (q[i]<=n)        //为第i个皇后找到了一个合适位置(i,q[i])
    {  if (i==n)        //若放置了所有皇后,输出一个解
        dispasolution(n);
      else              //皇后没有放置完
      {  i++;           //转向下一行,即开始下一个新皇后的放置
        q[i]=0;        //每个新考虑的皇后初始位置置为0列
      }
    }
    else i--;          //若第i个皇后找不到合适的位置,则回溯到上一个皇后
  }
}
```

```

bool place(int i)      //测试第i行的q[i]列上能否摆放皇后
{
    int j=1;
    if (i==1) return true;
    while (j<i)        //j=1~i-1是已放置了皇后的行
    {
        if ((q[j]==q[i]) || (abs(q[j]-q[i])==abs(j-i)))
            //该皇后是否与以前皇后同列，位置(j,q[j])与(i,q[i])是否同对角线
            return false;
        j++;
    }
    return true;
}

```

算法分析: 该算法中每个皇后都要试探 n 列，共 n 个皇后，其解空间是一棵子集树，不同于前面一般的二叉树子集树，这里每个结点可能有 n 棵子树。对应的算法时间复杂度为 $O(n^n)$ 。

实验5-3: n皇后问题(回溯法)

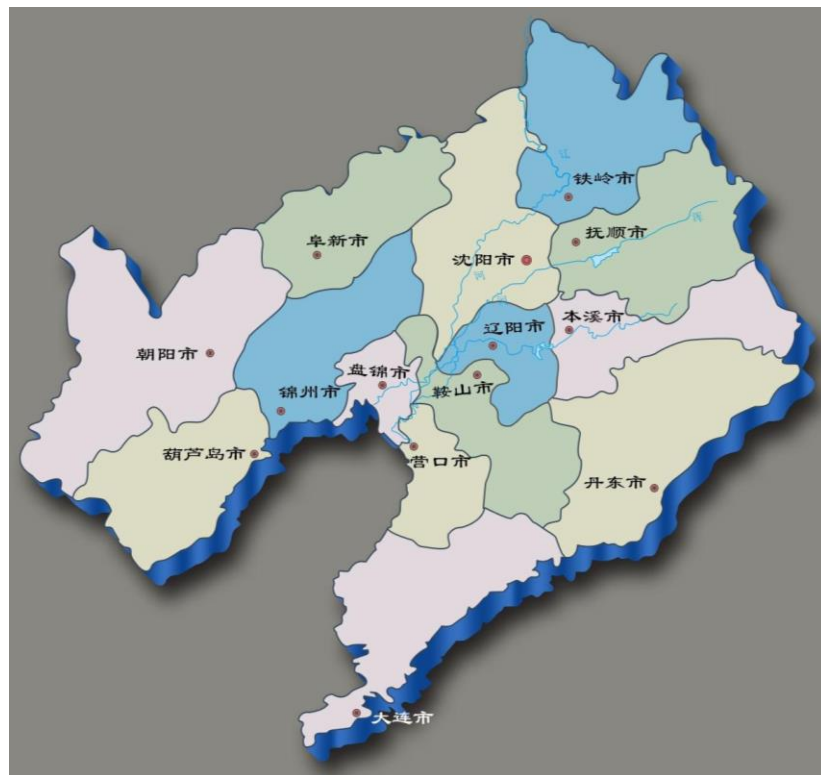
- 问题描述:在 $n \times n$ 的棋盘上摆放 n 个皇后, 使其不能互相攻击, 即任意两个皇后都不能处于同一行、同一列或同一斜线上。
- 示例 :
- 要求:
 - 1 填空完成程序;
 - 2 输出每个解 n 个皇后的位置, 给出解的个数。
 - *3 分析解空间的分支总数, 利用程序计算回溯法生成的分支数。
 - *4 报告中指出是排列树还是子集树。

5.6 图的m着色问题

图的着色问题是由地图的着色问题引申而来的：用 m 种颜色为地图着色，使得地图上的每一个区域着一种颜色，且相邻区域颜色不同。

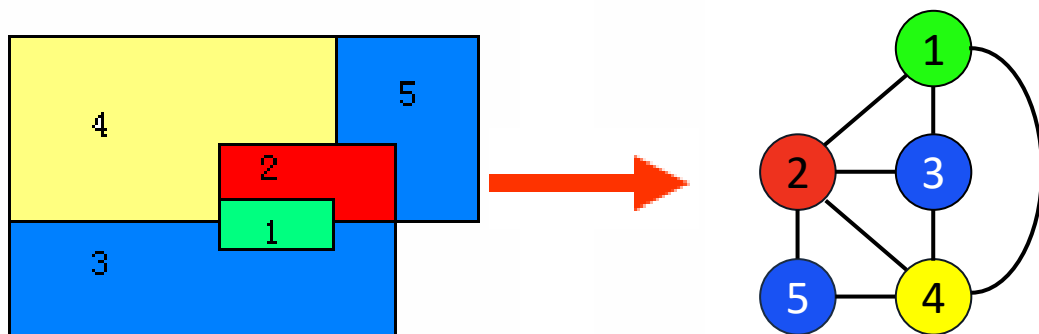
问题处理：如果把每一个区域收缩为一个顶点，把相邻两个区域用一条边相连接，就可以把一个区域图抽象为一个平面图。

19世纪50年代，英国学者提出了任何地图都可以4种颜色来着色的4色猜想问题。过了100多年，这个问题才由美国学者在计算机上予以证明，这就是著名的四色定理。



5.6 图的m着色问题

- 图着色问题描述为：给定无向连通图 $G=(V, E)$ 和正整数 m ，求最小的整数 m ，使得用 m 种颜色对 G 中的顶点着色，使得任意两个相邻顶点着色不同。这个问题是图的 m 可着色判定问题。
- 若一个图最少需要 m 种颜色才能使图中每条边连接的2个顶点着不同颜色，则称这个数 m 为该图的色数。求一个图的色数 m 的问题称为图的 m 可着色优化问题。



5.6 图的 m 着色问题

- 由于用 m 种颜色为无向图 $G=(V, E)$ 着色，其中， V 的顶点个数为 n ，可以用一个 n 元组 $C=(c_1, c_2, \dots, c_n)$ 来描述图的一种可能着色，其中， $c_i \in \{1, 2, \dots, m\} (1 \leq i \leq n)$ 表示赋予顶点 i 的颜色。
- 例如，5元组 $(1, 2, 2, 3, 1)$ 表示对具有5个顶点的无向图的一种着色，顶点1着颜色1，顶点2着颜色2，顶点3着颜色2，如此等等。
- 如果在 n 元组 C 中，所有相邻顶点都不会着相同颜色，就称此 n 元组为可行解，否则为无效解。

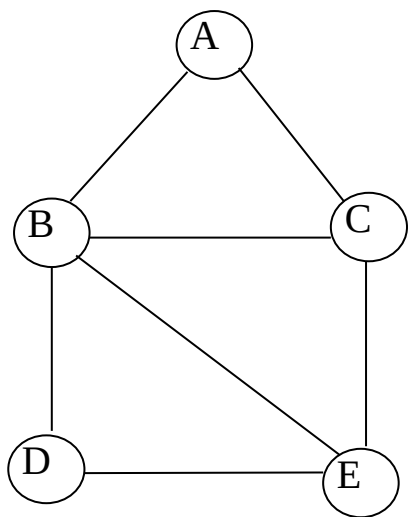
5.6 图的m着色问题

回溯法求解图着色问题:

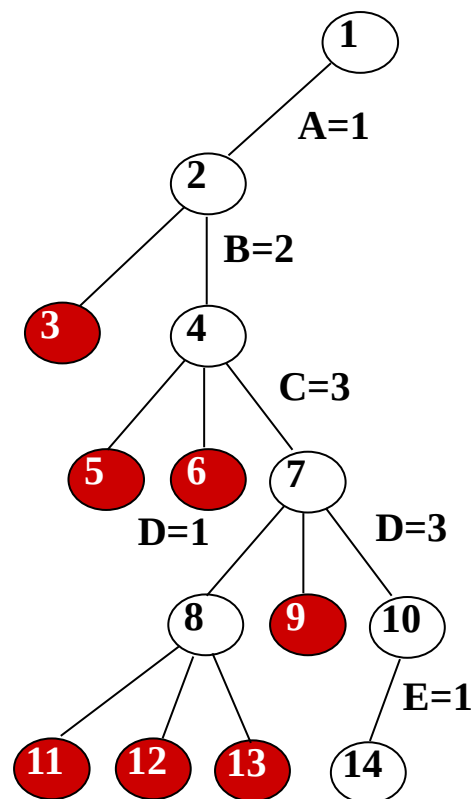
- 所有顶点的颜色初始化为0，依次着色。如果顶点 i 着色，且与相邻两个顶点的颜色不同，则当前着色是有效的局部着色；否则，为无效着色。
- 如果由根节点到当前节点路径上的着色，对应于一个有效着色，并且路径的长度小于 n ，那么相应的着色是有效的局部着色。则从当前节点出发，探索它的儿子节点，并把儿子结点标记为当前结点。如果在相应路径上搜索不到有效的着色，就把当前结点标记为 $d_$ 结点，去搜索对应于另一种颜色的兄弟结点。
- 如果对所有 m 个兄弟结点，都搜索不到一种有效着色，就回溯到它的父亲结点，并把父亲结点标记为 $d_$ 结点，转移去搜索父亲结点的兄弟结点。搜索过程一直进行，直到根结点变为 $d_$ 结点，或者搜索路径长度等于 n ，并找到了一个有效的着色为止。

5.6 图的m着色问题

回溯法求解图着色问题示例



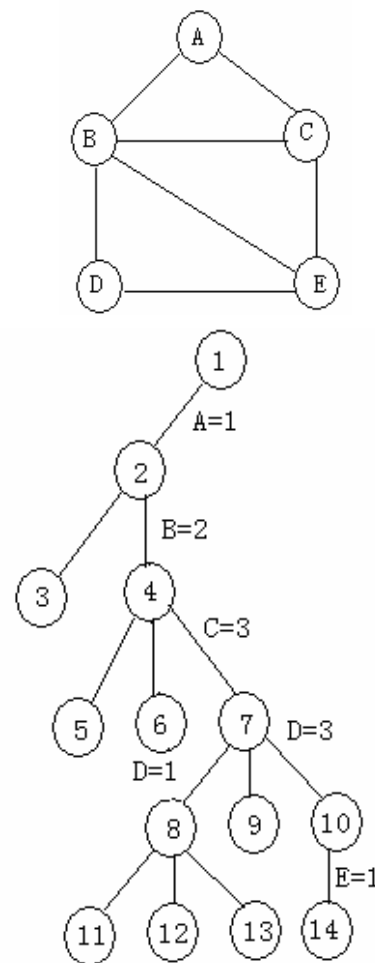
(a) 一个无向图



(b) 回溯法搜索空间

5.6 图的m着色问题

- ① 把5元组初始化为 $(0,0,0,0,0)$ ，从根结点开始向下搜索，以颜色1为顶点A着色，生成结点2时，产生 $(1,0,0,0,0)$ ，是个有效着色。
- ② 以颜色1为顶点B着色生成结点3时，产生 $(1,1,0,0,0)$ ，是个无效着色，结点3为d_结点。
- ③ 以颜色2为顶点B着色生成结点4，产生 $(1,2,0,0,0)$ ，是个有效着色。
- ④ 分别以颜色1和2为顶点C着色生成结点5和6，产生 $(1,2,1,0,0)$ 和 $(1,2,2,0,0)$ ，都是无效着色，因此结点5和6都是d_结点。
- ⑤ 以颜色3为顶点C着色，产生 $(1,2,3,0,0)$ ，是个有效着色。重复上述步骤，最后得到有效着色 $(1,2,3,3,1)$ 。



5.6 图的m着色问题

设数组color[n]表示顶点的着色情况，回溯法求解m着色问题的算法如下：

1. 将数组color[n]初始化为0;
2. $k=0$;
3. while ($k \geq 0$)
 - 3.1 依次考察每一种颜色，若顶点k的着色与其他顶点的着色不发生冲突，则转步骤3.2；否则，搜索下一个颜色；
 - 3.2 若顶点已全部着色，则输出数组color[n]，返回；
 - 3.3 否则，
 - 3.3.1 若顶点k是一个合法着色，则 $k=k+1$ ，转步骤3处理下一个顶点；
 - 3.3.2 否则，重置顶点k的着色情况， $k=k-1$ ，转步骤3。

GraphColor(int n,int m,int color[],bool c[][5])

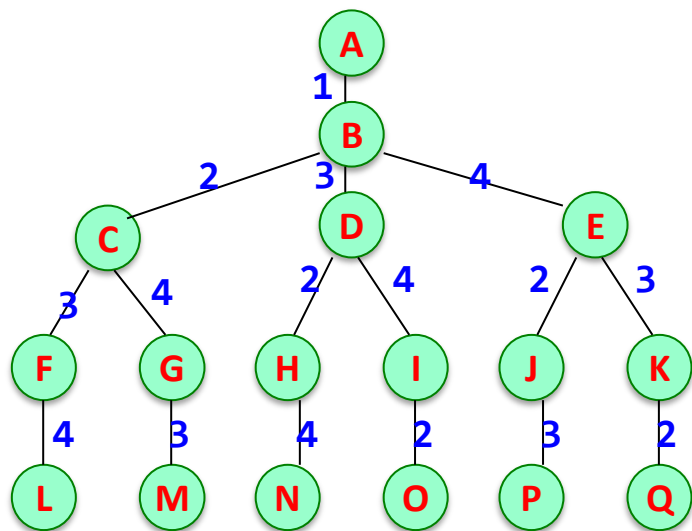
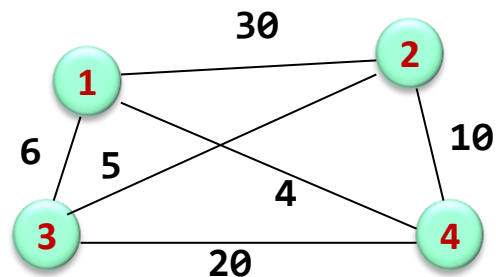
```
{
    int i,k;
    for (i=0; i<n; i++ )           //将解向量color[n]初始化为0
        color[i]=0;
    k=0;
    while (k>=0)
    { color[k]=color[k]+1;    //使当前颜色数加1
        while ((color[k]<=m) && (!ok(color,k,c,n))) //当前颜色是否有效
            color[k]=color[k]+1; //无效，搜索下一个颜色
        if (color[k]<=m)      //求解完毕，输出解
        {
            if (k==n-1)break; //是最后的顶点，完成搜索
            else k=k+1;      //否，处理下一个顶点
        }
        else                  //搜索失败，回溯到前一个顶点
        {
            color[k]=0;
            k=k-1;
        }
    }
}
```

5.7 TSP问题(旅行商、旅行售货员、货郎担)

问题描述：某售货员要到 n 个城市去推销商品，已知各城市之间的路程,请问他应该如何选定一条从城市1出发，经过每个城市一遍，最后回到城市1的路线，使得总的周游路程最小？

回溯法分析：该问题是一个NP完全问题，有 $(n-1)!$ 条可选路线。

实例：最优解(1,3,2,4,1)，最优值25



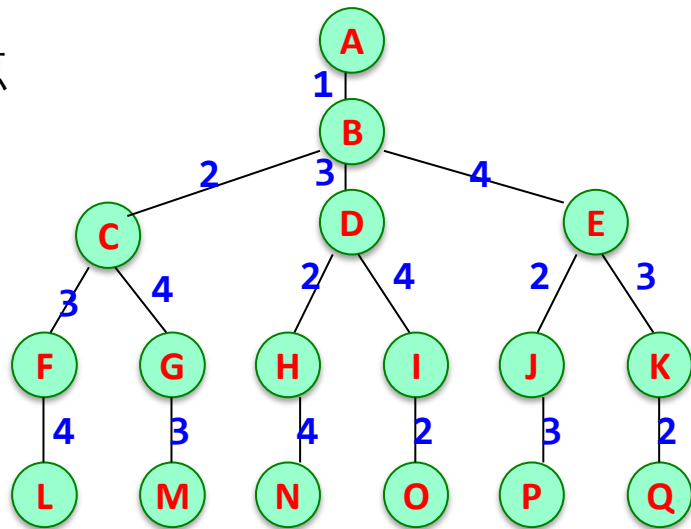
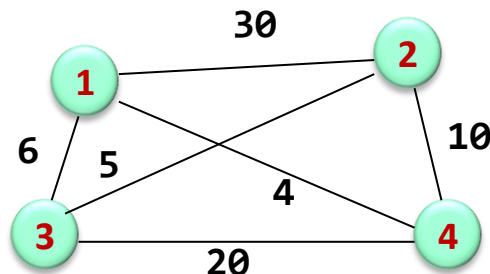
5.7 TSP问题(旅行商、旅行售货员、货郎担)

- **问题解空间**: 用图的邻接矩阵 a 表示无向连通图 $G = (V, E)$ 。若 (i, j) 属于图的边集, 则 $a[i][j]$ 表示两个城市之间的距离, 若 $a[i][j] = 0$, 表示城市之间无通路。问题的解空间就是出发城市+其他城市全排列。
- **解空间结构**: 为一颗排列树。
- **搜索方式**: 深度优先搜索
- **边界条件**:

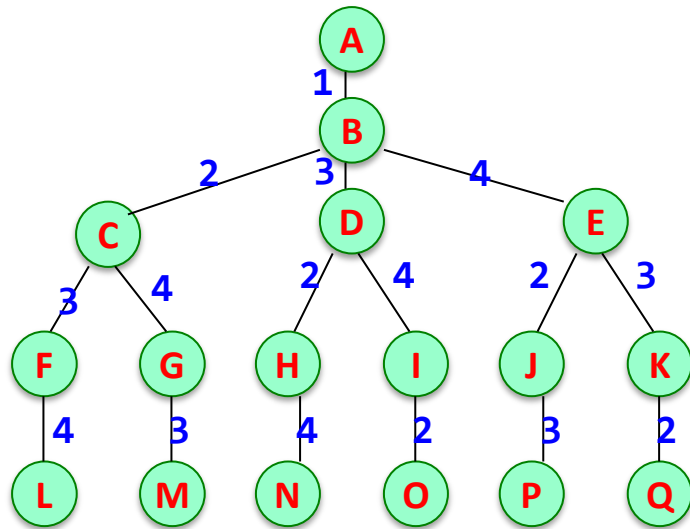
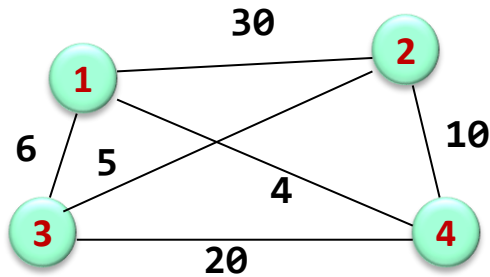
(1) 当 $i = n$ 时, 当前扩展的节点是排列树叶节点的父节点, 在此时需要检测是否存在一条从 $x[i-1]$ 到顶点 $x[n]$ 的边和一条 $x[n]$ 到 $x[1]$ 的边, 如果都存在, 则找到一条旅行售货员回路。并且还需要判断这条回路费用是否优于当前最优回路费用, 如果是, 必须进行更新。

(2) 当 $i < n$ 时, 如果存在路径并且最优值更小, 则进行更新, 否则剪去相应的子树。

算法设计



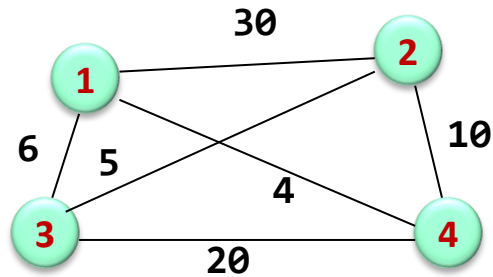
5.7 TSP问题(旅行商、旅行售货员、货郎担)



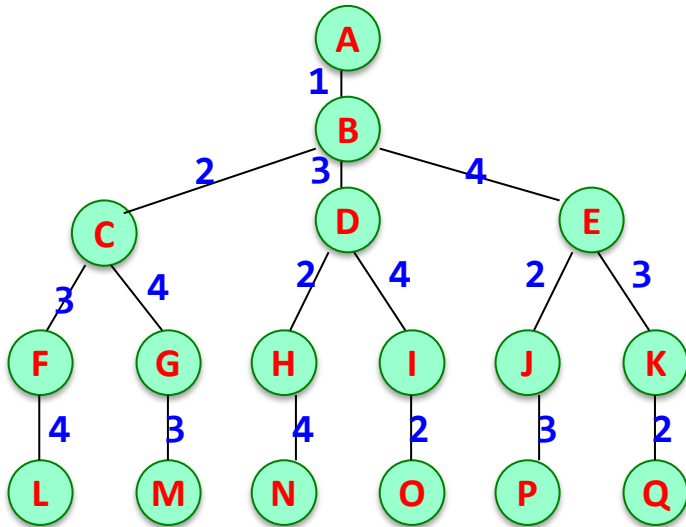
— 回溯算法将用深度优先方式从根节点开始，通过搜索解空间树发现一个**最小耗费的旅行**。

- 一个可能的搜索为 ABCFL。L点，旅行**1-2-3-4-1**作为当前最好的旅行被记录下来，耗费 **59**。
- 从L点回溯到活节点F，F没有未被检查的孩子，所以它成为死节点，回溯到 C点。
- C变为**E-节点**，向前移动到G，然后是M。这样构造出了旅行**1-2-4-3-1**，它的耗费是**66**。不比当前的最佳旅行好，**抛弃**它并**回溯**到G，然后是C,B。

5.7 TSP问题(旅行商、旅行售货员、货郎担)



- 从B点，搜索向前移动到D，然后是H,N。这个旅行1-3-2-4-1的耗费是25，比当前的最佳旅行好，把它作为当前的最好旅行。
- 从N点，搜索回溯到H，然后是D。在D点，再次向前移动，到达O点。
- 如此继续下去，可搜索完整棵树，得出1-3-2-4-1是最少耗费的旅行，耗费值为25。



5.7 TSP问题(旅行商、旅行售货员、货郎担)

算法实现

```
//问题表示
int n; //城市个数
int a[MAX][MAX]; //城市间距离,邻接矩阵, 0表示不通
//求解结果表示
int x[MAX]; //记录路径
int bestx[MAX] = { 0 }; //记录最优路径
int bestp = 63355; //最短路径长
int cp = 0; //当前路径长
```

5.7 TSP问题(旅行商、旅行售货员、货郎担)

算法实现

```
void backtrack(int t){
    if (t>n){//得到一个解
        if ((a[x[n]][1]) && (a[x[n]][1] + cp<bestp)){
            bestp = a[x[n]][1] + cp;
            for (int i = 1; i <= n; i++){
                bestx[i] = x[i];
            }
        }
    }
    else{
        for (int i = t; i <= n; i++){
            /*约束为当前节点到下一节点的长度不为0,限界为走过的长度+当前要走的长度之和小于
            最优长度*/
            if ((a[x[t - 1]][x[i]]) && (cp + a[x[t - 1]][x[i]]<bestp)){
                swap(x[t], x[i]);
                cp += a[x[t - 1]][x[t]];
                backtrack(t + 1);
                cp -= a[x[t - 1]][x[t]];
                swap(x[t], x[i]);
            }
        }
    }
}
```

5.8 求解活动安排问题

问题描述：假设有一个需要使用某一资源的 n 个活动所组成的集合 S ， $S=\{1, \dots, n\}$ 。该资源任何时刻只能被一个活动所占用，活动 i 有一个开始时间 b_i 和结束时间 e_i ($b_i < e_i$)，其执行时间为 $e_i - b_i$ ，假设最早活动执行时间为 0 。

一旦某个活动开始执行，中间不能被打断，直到其执行完毕。若活动 i 和活动 j 有 $b_i \geq e_j$ 或 $b_j \geq e_i$ ，则称这两个活动**兼容**。

设计算法求一种最优活动安排方案，使得**所有安排的活动个数最多**。

问题求解：这里采用回溯法求解，相当于找到 $S=\{1, \dots, n\}$ 的某个排列即调度方案，使得其中所有兼容活动的执行时间和最大，显然对应的解空间是一个排列树。

直接采用**排列树递归框架**实现，对于每一种调度方案求出所有兼容活动个数，通过比较求出最多活动个数，对应的调度方案就是最优调度方案，即为本问题的解。

对于一种调度方案，如何计算所有兼容活动的个数呢？因为其中可能存在不兼容的活动。

例如，有如表5.1所示的4个活动，若调度方案为（1，2，3，4），求所有兼容活动个数的过程如下：

活动编号	1	2	3	4
开始时间	1	2	4	6
结束时间	3	5	8	10

- ① 置当前活动最大结束时间 $laste=0$ ，所有兼容活动个数 $sum=0$ 。
- ② 活动1：其开始时间为1，大于等于 $laste$ ，属于兼容活动，选取它， sum 增加1， $sum=1$ ，置 $laste=$ 其结束时间=3。
- ③ 活动2：其开始时间为2，小于 $laste$ ，属于非兼容活动，不选取它。
- ④ 活动3：其开始时间为4，大于等于 $laste$ ，属于兼容活动，选取它， sum 增加1， $sum=2$ ，置 $laste=$ 其结束时间=8。
- ⑤ 活动4：其开始时间为6，小于 $laste$ ，属于非兼容活动，不选取它。
- ⑥ 该调度方案的所有兼容活动个数 sum 为2。

问题表示

```
struct Action
{   int b;           //活动起始时间
    int e;           //活动结束时间
};
int n=4;
Action A[]={0,0},{1,3},{2,5},{4,8},{6,10}}; //下标0不用
```

问题的求解结果表示:

```
int x[MAX];           //临时解向量
int bestx[MAX];        //最优解向量
int laste=0;          //一个调度方案中最后兼容活动的结束时间,初值为0
int sum=0;             //一个调度方案中所有兼容活动个数,初值为0
int maxsum=0;
```

```
void dfs(int i)           //搜索活动问题最优解
{  if (i>n)               //到达叶子结点,产生一种调度方案
    {  if (sum>maxsum)
        {  maxsum=sum;
            for (int k=1;k<=n;k++)
                bestx[k]=x[k];
        }
    }
}
```

```

else
{
    for(int j=i; j<=n; j++)
    {
        //第i层结点选择活动x[j]
        int sum1=sum;
        int laste1=laste;
        if (A[x[j]].b>=laste)
        {
            sum++;
            laste=A[x[j]].e;
        }
        swap(x[i],x[j]);
        dfs(i+1);
        swap(x[i],x[j]);
        sum=sum1;
        laste=laste1;
    }
}
}

```



活动编号	1	2	3	4
开始时间	1	2	4	6
结束时间	3	5	8	10

最优调度方案

选取活动1: [1, 3)

选取活动3: [4, 8)

安排活动的个数=2

算法分析: 该算法对应解空间树是一棵排列树, 与求全排列算法的时间复杂度相同, 即为 $O(n!)$ 。

5.9 求解流水作业调度问题

问题描述: 有 n 个作业（编号为 $1\sim n$ ）要在由两台机器M1和M2组成的流水线上完成加工。每个作业加工的顺序都是先在M1上加工，然后在M2上加工。M1和M2加工作业 i 所需的时间分别为 a_i 和 b_i （ $1\leq i\leq n$ ）。

流水作业调度问题要求确定这 n 个作业的最优加工顺序，使得从第一个作业在机器M1上开始加工，到最后一个作业在机器M2上加工完成**所需的时间最少**。可以假定任何作业一旦开始加工，就不允许被中断，直到该作业被完成，即**非优先调度**。

输入格式：输入包含若干个用例。每个用例第一行是作业数 n

($1 \leq n \leq 1000$)，接下来 n 行，每行两个非负整数，第 i 行的两个整数分别表示在第 i 个作业在第一台机器和第二台机器上加工时间。以输入 $n=0$ 结束。

输出格式：每个用例输出一行，表示采用最优调度所用的总时间，即从第一台机器开始到第二台机器结束的时间。

输入样例

4

5 6

12 2

4 14

8 7

0

输出样例

33

作业编号	1	2	3	4
M1时间a	5	12	4	8
M2时间b	6	2	14	7

问题求解：采用回溯法求解，对应的解空间是一个**排列树**，相当于求出 n 个作业的一种排列使完成时间最少。

作业的编号是 $1\sim n$ ，用数组 $x[]$ 作为解向量即调度方案，即 $x[i]$ 表示第 i 顺序执行的作业编号，初始时数组 x 的元素分别是 $1\sim n$ ，最优解向量用 $bestx[]$ 存储，对应的最优调度时间用 $bestf$ 表示。

求作业的所有排列可以直接采用**排列树递归框架**实现。对于每一种调度方案求出其所有作业执行的总时间，通过比较求出最小的总时间，对应的调度方案就是最优调度方案，即为本问题的解。

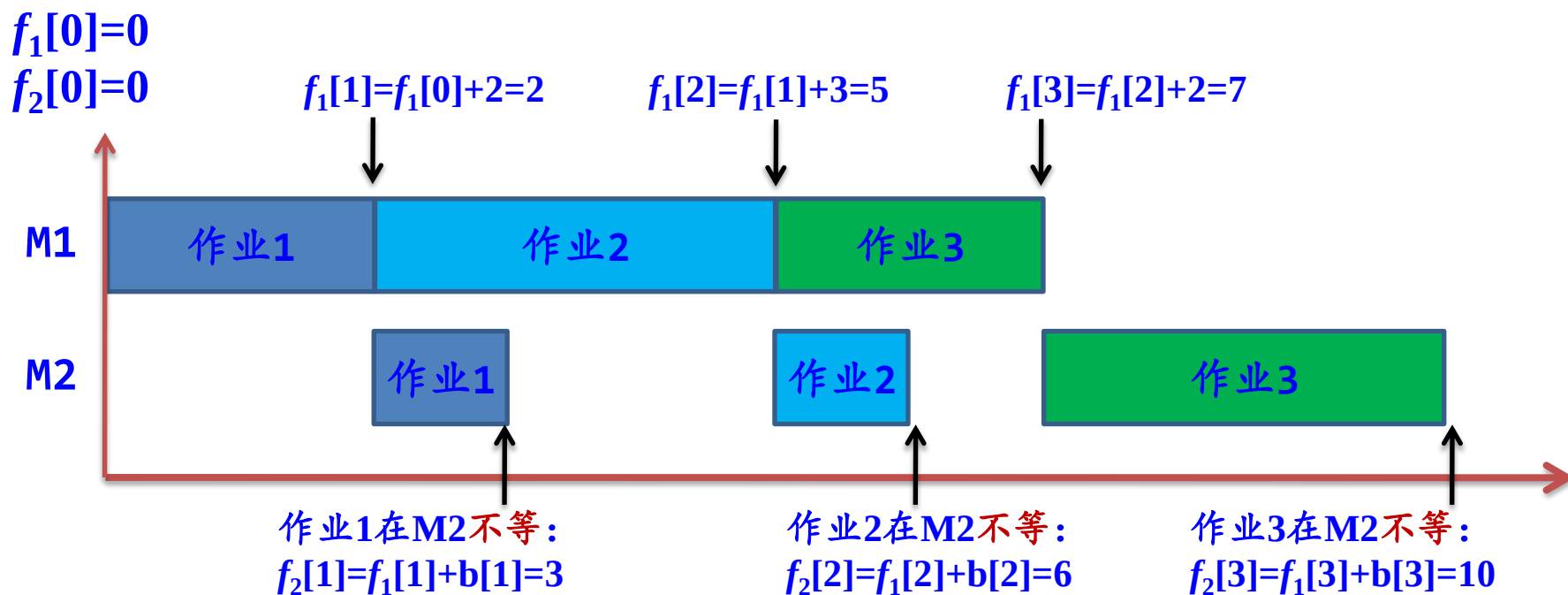
用 f_1 数组表示在M1上执行完当前作业 i 的总时间（含前面作业的执行时间）， f_2 数组表示在M2上执行完当前作业 i 的总时间（含前面作业的执行时间）。

由于一个作业总是先在M1上执行后在M2执行，所以 $f_2[n]$ 就是执行全部作业的总时间。

一个示例，假设有3个作业：

作业编号	1	2	3
M1时间a	2	3	2
M2时间b	1	1	3

现在的调用方案为 **(1, 2, 3)**，即按作业1、2、3的顺序执行。首先将 f_1 和 f_2 数组所有元素初始化为0。该调度方案的总时间计算：

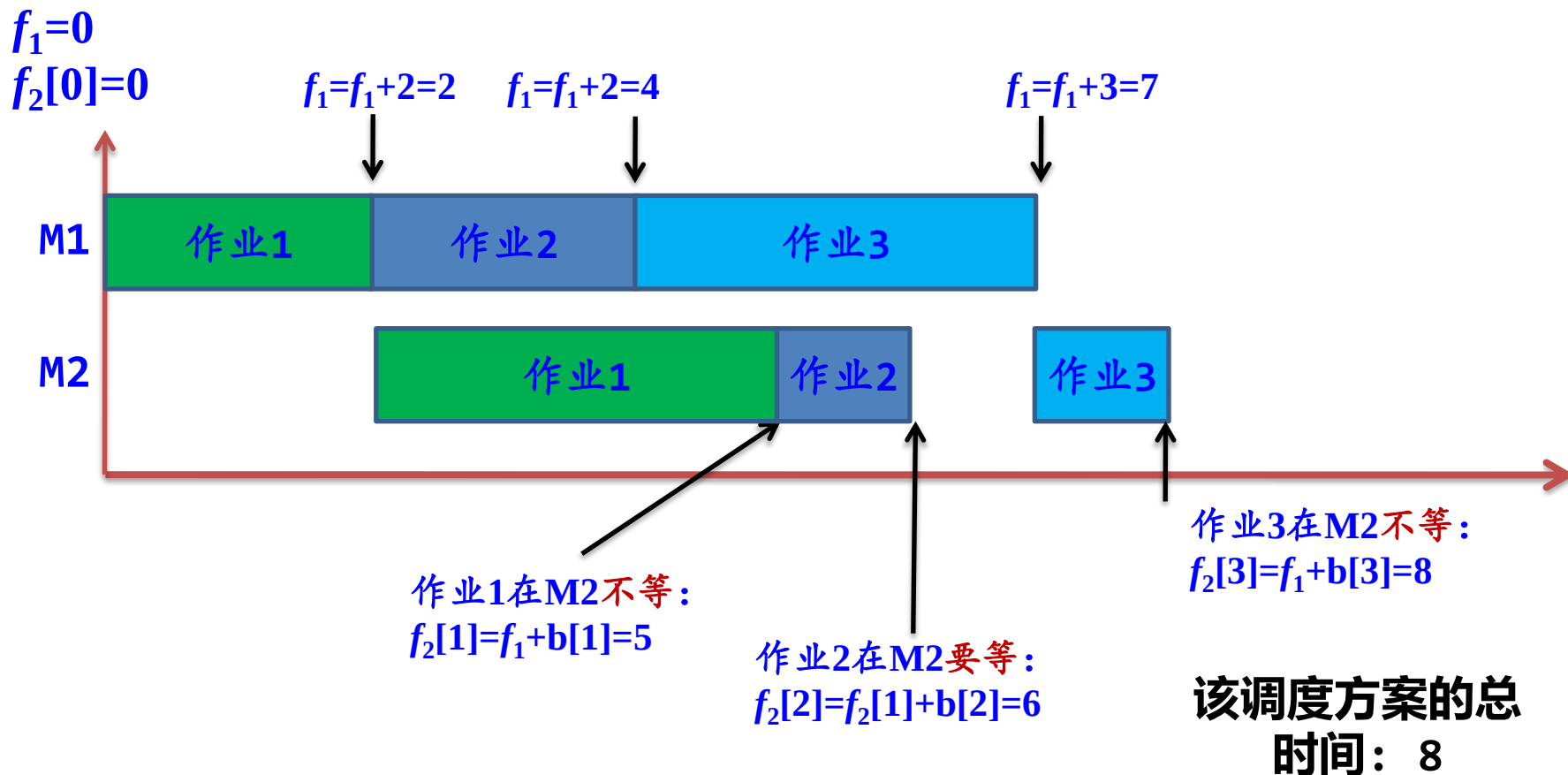


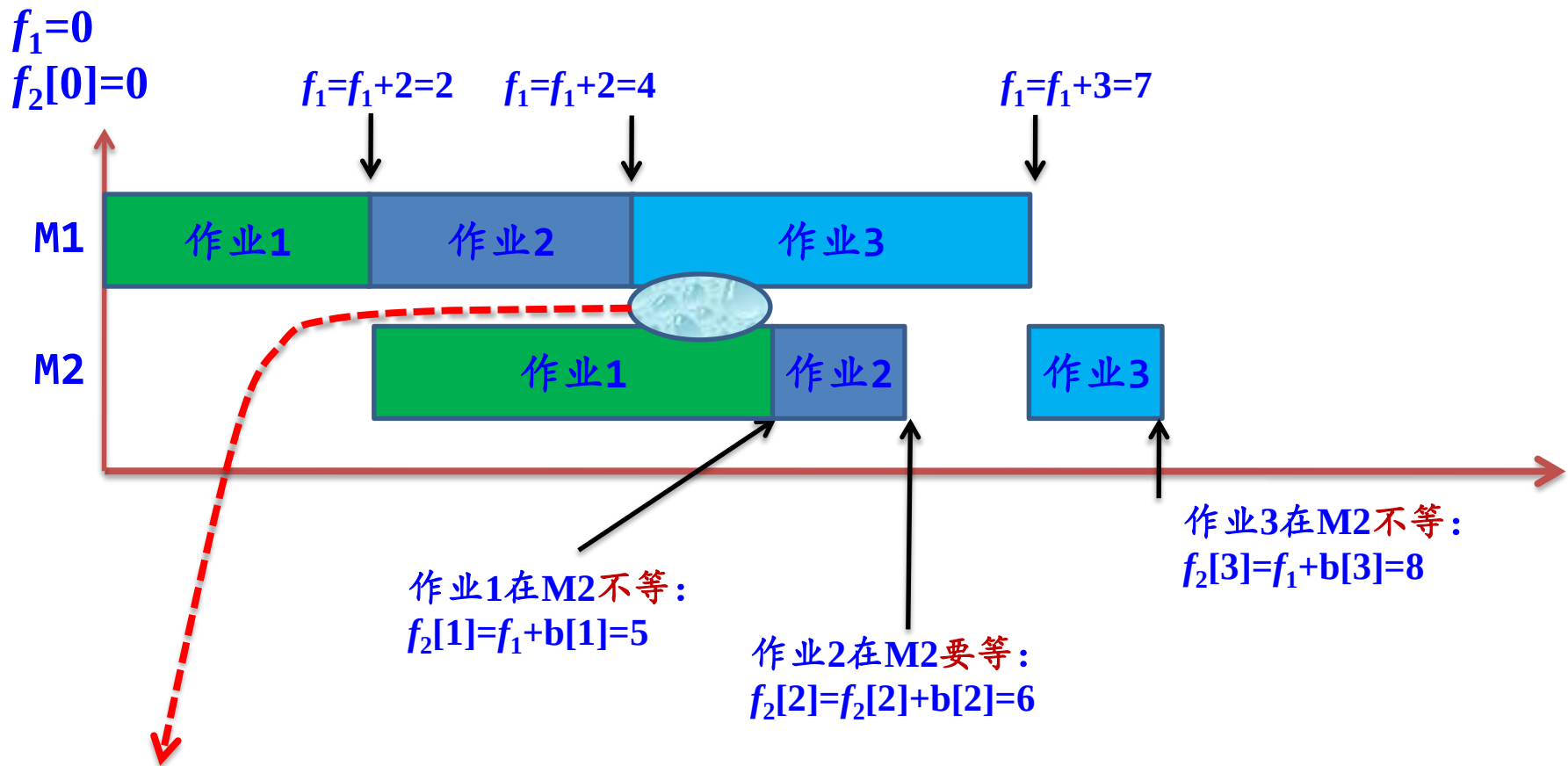
由于每个作业都是从M1开始的，即M1上各个作业是连续执行的，不需要等待，所以 f_1 不需要用数组表示，直接用单个变量 f_1 表示，

该调度方案的
总时间：10

再看看另外一种调用方案，假设3个作业如下表所示，调用方案仍然是(1, 2, 3)。该调度方案的总时间计算：

作业编号	1	2	3
M1时间a	2	2	3
M2时间b	3	1	1





当前作业*i*在M2上需要等待的条件: $f_2[i-1] > f_1$; 否则不需要等待

```

f1 += a[x[j]];           //在第i层选择执行作业x[j],在M1上执行完的时间
f2[i]=max(f1,f2[i-1])+b[x[j]];
  
```

```

void dfs(int i)                                //从第i层开始搜索
{
    if (i>n)                                    //到达叶子结点,产生一种调度方案
    {
        if (f2[n]<bestf)                        //找到更优解
        {
            bestf=f2[n];
            printf("    一个解: bestf=%d",bestf);
            printf(", 调度方案: "); disparr(x);
            printf(", f2: "); disparr(f2);
            printf("\n");
            for(int j=1; j<=n; j++)              //复制解向量
                bestx[j] = x[j];
        }
    }
}

```

```

else
{
    for(int j=i; j<=n; j++) //没有到达叶子结点,考虑i到n的作业
    {
        f1 += a[x[j]];
        //在第i层选择执行作业x[j],在M1上执行完的时间
        f2[i]=max(f1,f2[i-1])+b[x[j]];
        if (f2[i]<bestf)
            //剪枝:仅仅扩展当前总时间小于bestf的结点
        {
            swap(x[i],x[j]);
            dfs(i+1);
            swap(x[i],x[j]);
        }
        f1 -= a[x[j]];
        //回溯,即撤销第i层对作业x[j]的选择,以便再选择其他作业
    }
}
}

```

```
int n=4;  
int a[MAX]={0,5,12,4,8};  
int b[MAX]={0,6,2,14,7};
```

```
//作业数  
//M1上的执行时间,不用下标0的元素  
//M2上的执行时间,不用下标0的元素
```



作业编号	1	2	3	4
M1时间a	5	12	4	8
M2时间b	6	2	14	7

求解过程:

一个解: $bestf=42$, 调度方案: 1 2 3 4, $f2$: 11 19 35 42

一个解: $bestf=36$, 调度方案: 1 3 2 4, $f2$: 11 25 27 36

一个解: $bestf=34$, 调度方案: 1 3 4 2, $f2$: 11 25 32 34

一个解: $bestf=33$, 调度方案: 3 1 4 2, $f2$: 18 24 31 33

求解结果:

最少时间: 33, 最优调度方案: **3 1 4 2**

算法分析: 该算法的解空间树是一棵高度为 n 的**排列树**, 对应算法的时间复杂度为 $O(n!)$ 。